

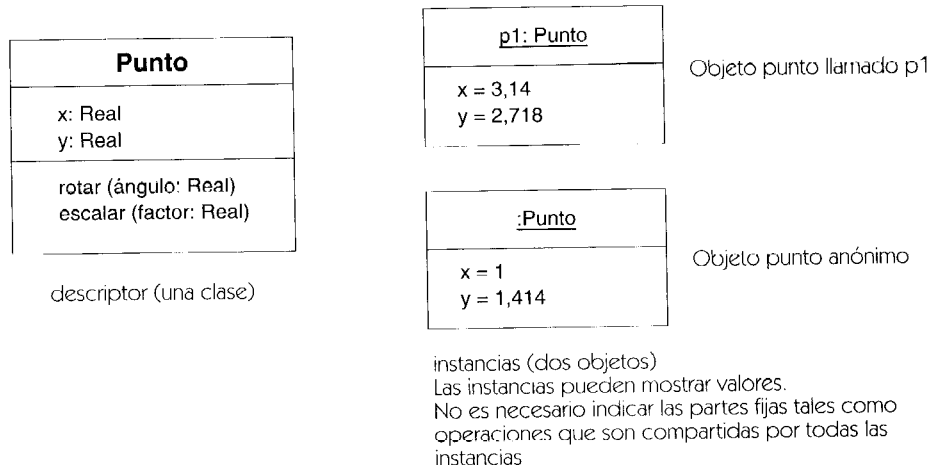


Figura 13.126 Descriptor e instancias

instanciable

Que puede poseer instancias. Es sinónimo de concreto.

Véase también abstracto, instancia directa, elemento generalizable.

Semántica

Los elementos generalizables se pueden crear como abstractos o como instanciables. Si son instanciables, entonces será posible crear instancias directas.

instanciación

Creación de nuevas instancias de elementos de un modelo.

Véase también iniciación.

Semántica

Las instancias se crean durante la ejecución como resultado de acciones primitivas de creación o de operaciones de creación. Primero se crea una identidad para la nueva instancia; después se reserva su estructura de datos según prescriba su descriptor; por último se dan valores iniciales a los valores de sus propiedades según prescriban el descriptor y el operador de creación.

La dependencia de utilización de la instanciación muestra la relación existente entre una operación que crea instancias o una clase que contiene esa operación y la clase de los objetos que se instancian.

Objetos. Cuando se instancia (se crea) un nuevo objeto, ese objeto se crea con una cierta identidad y una memoria, y luego se le dan valores iniciales. La iniciación de un objeto define los valores de sus atributos, sus asociaciones y su estado de control.

Normalmente, toda clase concreta posee una o más operaciones de constructor con alcance de clase cuyo propósito es crear nuevos objetos de la clase. Subyacente a todas las operaciones de constructor existe una operación primitiva implícita que crea una nueva instancia en blanco que después será iniciada por las operaciones de constructor. Una vez creada una instancia en blanco, ésta posee la forma prescrita por su descriptor, pero sus valores aún no han recibido valores iniciales, así que puede ser semánticamente inconsistente. Por tanto, una instancia no está disponible para el resto del sistema mientras no haya sido iniciada, lo cual sucederá inmediatamente después de la creación de la instancia en blanco.

Enlaces. Análogamente, los enlaces se crean mediante acciones u operaciones de creación, normalmente ejecutadas por operaciones de alcance de instancia asociadas a una de las clases participantes, en lugar de hacerlo mediante operaciones de constructor aplicadas al elemento de asociación en sí (aun cuando ésta es una posible técnica de implementación en algunas circunstancias). Una vez más, existe una operación primitiva implícita que crea un nuevo enlace entre una tupla específica de objetos. Esta operación no produce efectos si ya existe un enlace de la misma asociación entre la tupla de objetos (porque la extensión de una asociación es un conjunto y no puede contener valores duplicados). En una asociación ordinaria, ya no queda nada más por hacer. Un enlace de una clase asociación, sin embargo, requiere la inicialización de los valores de sus atributos.

Instancias de casos de uso. La instanciación de un caso de uso significa que se crea una instancia de caso de uso, y la instancia de caso de uso empieza a ejecutarse al principio del caso de uso que la controla. La instancia de caso de uso puede seguir temporalmente a otro caso de uso con el que esté relacionada por relaciones de extensión o inclusión antes de seguir ejecutando el caso de uso original. Cuando la instancia de caso de uso llega al final del caso de uso que está siguiendo, la instancia de caso de uso concluye.

Otras instancias. Las instancias de otros descriptores se pueden crear en un proceso análogo de dos pasos: primero se efectúa una creación en blanco para establecer la identidad y reservar la estructura de datos; luego se inician los valores de la nueva instancia para que cumpla todas las restricciones pertinentes. Por ejemplo, se crea implícitamente una activación como consecuencia directa de una llamada a una operación.

Los mecanismos exactos para la creación de instancias serán responsabilidad del entorno de ejecución.

Notación

La dependencia de instanciación se representa mediante una flecha discontinua que va desde la operación o clase que realiza la instanciación hasta la clase que se instancia; el estereotipo «**instantiate**» se asocia a la flecha.

Discusión

En algunas ocasiones se emplea el término instanciación para denotar el enlazado de una plantilla para producir un elemento enlazado, pero para esta relación es más específico el término enlazado.

instancia de

Es la relación entre una instancia y su descriptor.

Véase instancia.

instancia de caso de uso

Ejecución de una secuencia de acciones especificada en un caso de uso.

Véase caso de uso.

instancia directa

Una instancia, tal como un objeto, cuyo descriptor más específico, tal como una clase, es una clase dada.

Utilizado en una frase como, "El Objeto O es una instancia directa de la clase C." en este caso, la clase C es la clase directa del objeto O.

Véase clase directa.

instancia indirecta

Se trata de una entidad que es una instancia de un elemento, tal como una clase; además, es una instancia de un descendiente del elemento. Esto es, se trata de una instancia pero no es una instancia directa.

instanciar

Crear una instancia de un descriptor.

Véase instanciación.

instantánea

Colección de objetos, enlaces y valores que forma la configuración de un sistema en un determinado instante de su ejecución.

intención

Dícese de la especificación formal de las propiedades estructurales y de comportamiento de un descriptor. Compárese con: extensión.

Véase también descriptor.

Semántica

Un descriptor, tal como una clase o una asociación, posee tanto una descripción (su intención) como un conjunto de instancias que describe (su extensión). El propósito de la intención es especificar las propiedades estructurales y de comportamiento de las instancias en una forma ejecutable.

interacción

Se trata de la especificación de la forma en que se envían mensajes entre objetos u otras instancias para ejecutar una tarea. La interacción se define en el contexto de una colaboración.

Véase también diagrama de interacción.

Semántica

Los objetos u otras instancias de una colaboración se comunican para lograr un propósito (tal como llevar a cabo una operación) mediante el intercambio de mensajes. Entre los mensajes pueden contarse las señales y llamadas, así como interacciones más implícitas a través de condiciones o de eventos temporales. Un patrón de intercambios de mensajes que se realizan para lograr un propósito específico es lo que se denomina una interacción.

Estructura

Una interacción es una especificación de comportamiento que abarca una secuencia de intercambios de mensajes entre un conjunto de objetos, efectuada para lograr un cierto propósito tal como la implementación de una operación. Una interacción es una colaboración más un conjunto secuenciado de flujos de mensajes que se imponen a los enlaces de la colaboración. Para especificar una interacción, primero es necesario especificar una colaboración —esto es, definir los objetos que interaccionan y sus relaciones entre sí—. Entonces se especifican las posibles secuencias de interacción, bien en una única descripción que contiene condiciones (bifurcaciones o señales condicionales) o bien como múltiples descripciones, cada una de las cuales describe una de entre las posibles rutas de ejecución. La descripción completa del comportamiento de una colaboración se puede dar como una máquina de estados, cuyos estados son los estados de ejecución de una operación o de algún otro procedimiento.

Notación

Las interacciones se muestran como diagramas de secuencia o como diagramas de colaboración. Ambos formatos de diagrama muestran la ejecución de las colaboraciones. Los diagramas de secuencia son la visualización explícita del comportamiento de las colaboraciones, incluyendo la secuenciación temporal de mensajes y una representación explícita de las activaciones de métodos. Sin embargo, los diagramas de secuencias muestran únicamente los objetos participantes y no sus relaciones con otros objetos o sus atributos. Por tanto, no muestran en su to-

talidad el punto de vista contextual de una colaboración. Los diagramas de colaboración muestran todo el contexto de una interacción, incluyendo los objetos y sus relaciones relevantes para una interacción, así que suelen ser mejores que los diagramas de secuencia a efectos de diseño.

interfaz

Un conjunto de operaciones que posee un nombre y que caracteriza el comportamiento de un elemento.

Véase también clasificador, realización.

Semántica

Una interfaz es un descriptor de las operaciones visibles externamente de una clase u otra entidad (incluyendo las unidades de compendio, tales como los paquetes) que no especifica la estructura interna. Cada interfaz suele especificar únicamente una parte limitada del comportamiento de una clase real. Una clase puede admitir muchas interfaces, cuyos efectos podrán ser disjuntos o estar solapados. Las interfaces no poseen implementación; carecen de atributos, estados y asociaciones; sólo poseen operaciones y señales que reciben. Las interfaces pueden tener relaciones de generalización. Una interfaz descendiente incluye todas las operaciones y señales de sus antecesores pero puede añadir operaciones adicionales. En esencia, una interfaz equivale a una clase abstracta sin atributos ni métodos, que poseyera únicamente operaciones abstractas. Todas las operaciones de una interfaz tienen visibilidad pública (en caso contrario, no tendría sentido incluirlas, porque una interfaz no tiene nada “interno” que pueda hacer uso de ellas).

La siguiente definición extendida indica el propósito de una interfaz.

- Una interfaz es una colección de operaciones que se emplea para especificar un servicio de una clase o de un componente.
- Una interfaz sirve para nombrar una colección de operaciones y para especificar sus firmas y efectos. Una interfaz se centra en los efectos, no en la estructura, de un servicio dado. Una interfaz no ofrece una implementación para ninguna de sus operaciones. La lista de operaciones puede incluir también las señales que esa clase está dispuesta a manejar.
- Una interfaz se emplea para especificar un servicio que proporciona un proveedor y que pueden solicitar otros elementos. La interfaz da nombre a una colección de operaciones que funcionan en cooperación para realizar algún comportamiento de interés lógico de un sistema o como parte de un sistema.
- Una interfaz define un servicio que ofrece una clase o componente. Define un servicio que a su vez es implementado por una clase o componente. Como tal, una interfaz abarca los límites lógicos y físicos de un sistema. Una o más clases (que formarán probablemente parte de un subsistema componente) pueden proporcionar una implementación lógica de la interfaz. Uno o más componentes pueden proporcionar un empaquetamiento físico que se adapte a esa misma interfaz.

- Si una clase realiza (implementa) una interfaz, entonces debe declarar o heredar todas las operaciones de la interfaz. Si una clase realiza más de una interfaz, tiene que contener todas y cada una de las operaciones que se encuentren en cualquiera de sus interfaces. Una misma operación puede aparecer en más de una interfaz. Si coinciden sus firmas, deben representar la misma operación o bien estarán en conflicto y el modelo estará mal formado. (Una implementación puede adoptar reglas específicas de un lenguaje para las firmas coincidentes. Por ejemplo, en C++ los nombres de los parámetros y los tipos proporcionados se ignoran.) Una interfaz no hace afirmación alguna acerca de los atributos o asociaciones de una clase; éstos forman parte de su implementación.

Una interfaz es un elemento generalizable. Una interfaz descendiente hereda todas las operaciones de su predecesor y puede añadir algunas operaciones. La realización puede considerarse como una herencia de comportamiento; una clase hereda las operaciones de otro clasificador, pero no su estructura. Una clase puede realizar a otra clase. La clase que haga las veces de especificación actuará como interfaz en tanto en cuanto sólo sus operaciones afectan a la relación.

Las interfaces no participan en las asociaciones. Una interfaz no puede tener una asociación que sea navegable partiendo de la interfaz. Una interfaz, sin embargo, puede ser el destino de una asociación, siempre y cuando la asociación sólo sea navegable hacia la interfaz.

Notación

Una interfaz es un clasificador y se puede mostrar empleando el símbolo del rectángulo con la palabra reservada «**interface**». En el compartimento de operación se pone una lista de las operaciones que admite esa interfaz. Las señales que llevan el estereotipo «**signal**» se pueden incluir también en la lista de operaciones o bien se pueden enumerar en su propio compartimento. Se puede omitir el compartimento de atributos porque siempre está vacío.

También se puede visualizar una interfaz mediante un círculo con el nombre de la interfaz situado por debajo del símbolo. El círculo puede estar conectado mediante una línea continua con clases (u otros elementos) que lo admitan. También puede conectarse con contenedores de nivel superior, tales como los paquetes, que contengan las clases. Esto indica que la clase proporciona todas las operaciones del tipo de interfaz (y posiblemente más). La notación de círculo no muestra la lista de operaciones que admite la interfaz. Utilice el símbolo de rectángulo completo para mostrar la lista de operaciones. Una clase que utilice o requiera operaciones proporcionadas por la interfaz se puede conectar al círculo mediante una flecha discontinua que apunte al círculo. La flecha discontinua implica que la clase requiere las operaciones especificadas en la interfaz para algún propósito, pero no se exige que la clase cliente haga uso de *todas* las operaciones de la interfaz. Un servicio suele especificarse mediante una prueba de suficiencia. Si un proveedor proporciona las operaciones contenidas en un cierto conjunto de interfaces, entonces satisface los requisitos de los clientes.

La relación de realización se muestra mediante una línea discontinua con una cabeza de flecha triangular sólida (un “símbolo discontinuo de generalización”) que va desde una clase a una interfaz que admita ésta. Se trata de la misma notación que se emplea para indicar la realización de un tipo por parte de una clase de implementación. De hecho, este símbolo se puede emplear entre dos símbolos de clasificador, indicando que el cliente (situado en la cola de la flecha) admite todas las operaciones definidas en el proveedor (situado en la cabeza de la

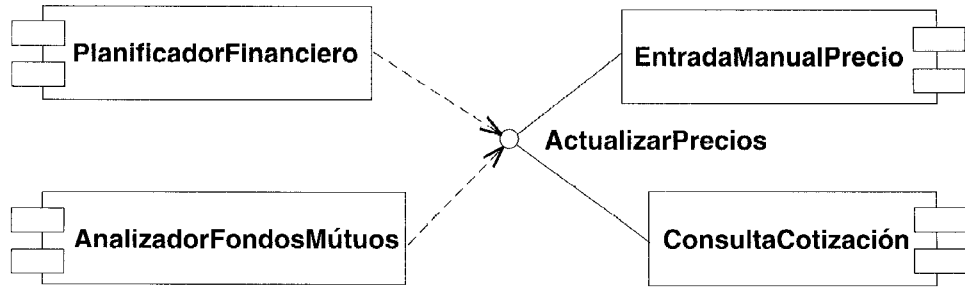


Figura 13.127 Proveedores y clientes de la interfaz

flecha), pero sin necesidad de admitir ninguna estructura de datos del proveedor (atributos y asociaciones).

Ejemplo

La Figura 13.127 muestra una vista simplificada de los componentes financieros que intervienen en los precios de las acciones de bolsa. El **PlanificadorFinanciero** es una aplicación financiera personal que administra nuestras inversiones, así como los gastos personales. El **AnalizadorFondosPensiones** examina detalladamente los fondos de pensiones. Necesita poseer la capacidad de actualizar los precios de los valores subyacentes, así como lo precios de los fondos. La capacidad de actualizar los precios de los valores se muestra mediante la interfaz **ActualizarPrecios**. Hay dos componentes que implementan esta interfaz; se muestran mediante las líneas continuas que los unen al símbolo de interfaz. El componente **EntradaManualPrecio** permite al usuario introducir manualmente los precios de los valores seleccionados. El componente **ConsultaCotización** recupera los precios de esos valores en un servidor bursátil, empleando un módem o Internet.

La Figura 13.128 muestra la notación completa de una interfaz como palabra reservada en un símbolo de clase. Se observará que esta interfaz implica dos operaciones —preguntar el precio de unas acciones y conseguir una cotización—; y además proporcionar una lista de valores de bolsa y recibir una lista de los precios que hayan cambiado. En este diagrama el componente **ConsultaCotización** está conectado a la interfaz empleando una flecha de realización, pero se trata de la misma relación que se mostraba en el diagrama anterior; sólo se está empleando una notación más explícita.

Este diagrama muestra también una nueva interfaz, **Actualizar Periódica Precios**, que es un descendiente de la interfaz original. Hereda las dos operaciones y agrega una tercera operación que envía una solicitud de actualización periódica y automática de los precios. Esta interfaz es realizada por el componente **ServidorCotización**, un servicio de suscripción. Implementa las dos mismas operaciones que **ConsultaCotización** pero de una manera diferente. No comparte la implementación de **ConsultaCotización** (en este ejemplo) y por tanto no hereda de él una implementación.

La Figura 13.128 muestra la diferencia entre herencia de interfaz y herencia completa. Esa última implica a la primera, pero una interfaz descendiente puede ser implementada de una forma distinta de la interfaz predecesora. **ServidorCotización** admite la interfaz que implementa

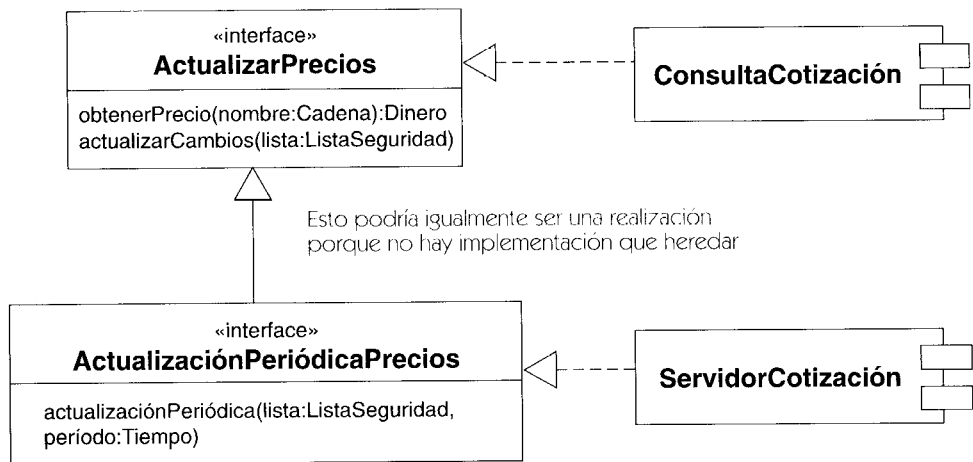


Figura 13.128 La notación de interfaz completa

ConsultaCotización, a saber, **ActualizarPrecios**, pero no hereda la implementación de **ConsultaCotización**. (En general, resulta cómodo heredar la implementación además de la interfaz, así que las dos jerarquías suelen ser idénticas.)

Una interfaz también puede contener una lista de las señales que maneja (Figura 13.129).

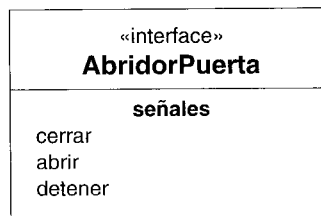


Figura 13.129 Una interfaz con señales

Las interfaces se emplean para definir el comportamiento de las clases, así como el de los componentes, sin restringir la implementación. Esto permite distinguir la herencia de interfaz, tal como se declara en Java, de la herencia de implementación.

invariante

Una restricción que forzosamente ha de ser cierta en todo momento (o por lo menos en todo momento en que no haya ninguna operación sin finalizar).

Semántica

El invariante es una expresión booleana que debe ser cierta siempre que no haya ninguna operación activa. Se trata de una aserción y no de una sentencia ejecutable. Dependiendo de la

forma exacta de la expresión, puede ser o no posible verificarla automáticamente por anticipado.

Véase también precondition, postcondición.

Estructura

Los invariantes se modelan como una restricción con el estereotipo «**invariant**» asociado al elemento.

Notación

Se puede mostrar una postcondición en una nota con la palabra reservada «**invariant**». La nota está asociada a un clasificador, atributo u otro elemento.

ligadura

Asignación de valores a parámetros para producir un elemento individual a partir de un elemento parametrizado. La relación de ligadura es un tipo de dependencia, que se utiliza para ligar plantillas y así producir nuevos elementos del modelo.

Véase también elemento ligado, plantilla.

Semántica

Una definición parametrizada, como una operación, señal o plantilla, define la forma de un elemento. Un elemento parametrizado no se puede utilizar directamente, ya que sus parámetros no tienen valores específicos. La ligadura es una dependencia que representa la asignación de valores a parámetros para producir un nuevo elemento que pueda ser utilizado. La ligadura actúa sobre las operaciones para producir llamadas, sobre las señales para producir señales de envío y sobre las plantillas para producir nuevos elementos del modelo. En las dos primeras, la ligadura se produce durante la ejecución para producir entidades en tiempo de ejecución que normalmente no figuran en los modelos excepto como ejemplos o resultados de la ejecución. Los valores de los argumentos se definen dentro del sistema de ejecución.

Sin embargo, una plantilla se liga en tiempo de modelado para producir nuevos elementos utilizables en el modelo. Los valores de los argumentos pueden ser otros elementos, como clases, además de valores dato, como cadenas o enteros. La relación de ligadura liga valores a una plantilla produciendo un elemento real del modelo que puede utilizarse directamente dentro del mismo.

Una relación de ligadura tiene un elemento proveedor (la plantilla), un cliente (el elemento recién generado), y una lista de valores para ligarlos con los parámetros de la plantilla. El

elemento ligado se define sustituyendo cada parámetro por el valor del correspondiente argumento en una copia del cuerpo de la plantilla. La clasificación de cada argumento debe ser la misma o un descendiente de la clasificación declarada de su parámetro.

Las ligaduras no afectan a la plantilla, pues se puede ligar cada plantilla muchas veces para producir cada vez un nuevo elemento ligado.

Notación

La ligadura se indica mediante la palabra clave «**bind**» asociada a una flecha discontinua que conecta el elemento generado (en el origen de la flecha) con la plantilla (en la punta de la flecha). Los valores reales de los argumentos se representan mediante una lista de expresiones de texto separadas por comas encerrada entre paréntesis a continuación de la palabra «**bind**».

Una notación alternativa y más compacta de la ligadura emplea la correspondencia de nombres para evitar la necesidad de flechas. Para representar un elemento generado mediante ligadura, se añade al nombre de la plantilla una lista de expresiones de texto separadas por comas encerrada entre los símbolos menor y mayor que (<argumento_{lista}>).

En cualquier caso, cada argumento se representa mediante una cadena de texto que se evalúa estáticamente en el momento de construir el modelo. No se evalúa dinámicamente como las operaciones o los argumentos de tipo señal.

En la Figura 13.130 se declara explícitamente (utilizando flechas) una nueva clase **ListaDirecciones**, cuyo nombre puede utilizarse en modelos y expresiones. La forma implícita **Farray<Punto,3>** declara una «clase anónima» que no tiene nombre por sí misma, y que puede utilizarse en expresiones utilizando la sintaxis en línea. En ningún caso se pueden declarar atributos u operaciones adicionales: si se necesitan extensiones debe declararse una subclase.

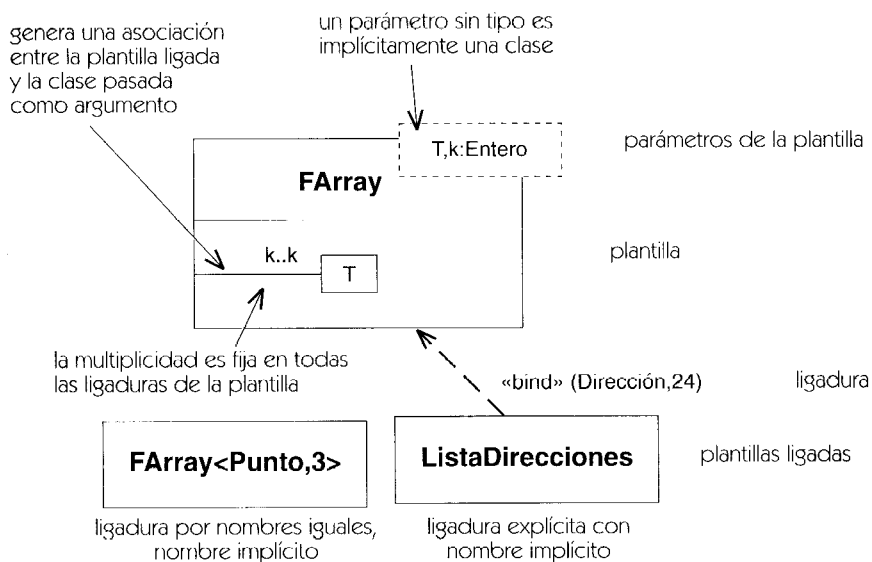


Figura 13.130 Plantillas: declaración y ligadura

Elementos estándar

bind (ligar).

línea de vida

Denota una línea discontinua que aparece en los diagramas de secuencia y muestra la existencia de un objeto a lo largo de un cierto período de tiempo. La línea es paralela al eje temporal.

Véase también diagrama de secuencia.

Semántica

La línea de vida indica el período durante el cual existe un objeto. Un objeto está activo si posee un hilo de control; esto es, durante el período de tiempo en el cual posee una llamada a procedimiento que no ha concluido. Esto último se denomina una activación. Incluye el tiempo durante el cual el procedimiento está llamando a un procedimiento de nivel inferior.

Notación

Los objetos o roles de clasificador se muestran en los diagramas de secuencia en forma de una línea vertical, llamada línea de vida. La línea de vida representa la existencia del objeto en un determinado momento.

Las flechas entre líneas de vida denotan mensajes entre objetos. Una flecha que tiene la punta en una línea de vida es un mensaje que recibe el objeto, una operación de la cual es responsable; una flecha que tiene su origen en una línea de vida es un mensaje que ha enviado el objeto, una operación invocada por él. El orden geométrico de las flechas de mensajes a lo largo de la línea de vida indica el orden temporal relativo de los mensajes.

Si el objeto es creado o destruido durante el período de tiempo que se muestra en el diagrama, entonces su línea de vida comienza o se detiene en el punto correspondiente. En caso contrario, progresa desde la parte superior del diagrama hasta la parte inferior del mismo. En la cabeza de la línea de vida se dibuja el símbolo de un objeto. Si el objeto se crea durante el período de tiempo que se muestra en el diagrama, entonces el símbolo del objeto se dibuja en la punta del mensaje que lo crea. En caso contrario, se dibuja el símbolo del objeto por encima de todas las flechas de mensajes. Si se destruye el objeto durante el período de tiempo que muestra el diagrama, entonces su destrucción se marca mediante una X de gran tamaño, bien en la punta de flecha del mensaje que produce la destrucción o bien (en el caso de una autodestrucción) en el mensaje final que proporciona el objeto destruido. Los objetos que existen cuando comienza una transición se muestran en la parte superior del diagrama (por encima de la primera flecha). Un objeto que existe cuando finaliza la transición posee una línea de vida que sigue adelante más allá de la flecha final.

La línea de vida puede escindirse en dos o más líneas de vida concurrentes que se mostrarán condicionalmente. Cada pista corresponde a una bifurcación condicional en el flujo de men-

sajes. Las líneas de vida pueden volver a unirse en algún punto posterior. Véase la Figura 13.70 para observar un ejemplo. Esta notación puede inducir a confusión y debería utilizarse con parquedad.

El período de tiempo durante el cual está activo temporal o permanentemente el objeto se muestra mediante una línea doble continua que se superpone para mostrar recursividad. Como un objeto activo siempre está activo, la línea doble se omite en ciertas ocasiones, porque no añade información.

Las líneas de vida pueden verse interrumpidas por un símbolo de estado, para mostrar un cambio de estado. Esto se corresponde con una transición de conversión en el seno de un diagrama de colaboración. Se puede dibujar una flecha hasta el símbolo de estado para indicar el mensaje que ha dado lugar al cambio de estado. Véase la Figura 13.71 como ejemplo.

línea de vida del objeto

Denota una línea de un diagrama de secuencia que representa la existencia de un objeto durante un cierto período de tiempo.

lista

Una colección ordenada, de longitud variable y formada por elementos del modelo, que pertenece a otro elemento del modelo, estando anidada en él.

Véase también clasificador, estado.

Semántica

Los clasificadores contienen varias listas de elementos subordinados, que incluyen atributos, operaciones y métodos. Un estado contiene una lista de transiciones internas. Hay otras clases de elementos que contienen listas de otros tipos de elementos. Cada clase de lista se describe individualmente. Esta entrada describe las propiedades de las listas empotradas, en general. Además de las listas de atributos y de operaciones, hay listas opcionales que pueden mostrar valores predefinidos o definidos por el usuario, tales como responsabilidades, reglas o historias de modificaciones. UML no define estas listas opcionales. La manipulación de las listas definidas por el usuario depende de las herramientas.

Una lista empotrada y los elementos de la lista pertenecen exclusivamente a la clase que la contiene o a algún elemento contenedor. La propiedad no se comparte entre múltiples contenedores. Las otras clases pueden tener posibilidad de acceder a los elementos de la lista —por ejemplo, por herencia o asociación— pero la propiedad de las listas contenidas para modificar el modelo pertenece al contenedor inmediato. Los elementos poseídos se almacenan, se copian y se destruyen junto con sus contenedores.

Los elementos de una lista tienen un orden que está determinado por quien haga el modelado. El orden puede resultar útil para el modelador —por ejemplo, se puede emplear en un ge-

nerador de código para generar una lista de declaraciones en algún lenguaje de programación—. Si al modelador no le preocupa el orden, quizá porque el modelo se encuentra en la fase de análisis o porque el lenguaje ignora el orden, el orden seguirá existiendo en el modelo pero puede ignorarse sencillamente por ser irrelevante.

Notación

Las listas empotradas aparecen dentro de su propio compartimento como listas de cadenas, habiendo una cadena por línea para cada elemento de la lista. Cada cadena es la representación codificada de una característica, tal como un atributo, una operación, una transición interna y así sucesivamente. La naturaleza de la codificación se describe en el artículo correspondiente a cada clase de elemento.

Orden. El orden canónico de las cadenas es el mismo que tiene la lista de elementos dentro del modelo, pero el orden interno se puede anular opcionalmente y se pueden ordenar las cadenas de acuerdo con alguna propiedad interna, tal como el nombre, visibilidad o estereotipo. Sin embargo, observe que los elementos mantienen su orden original en el modelo subyacente. La información relativa sólo se suprime de la visualización.

Puntos suspensivos. Unos puntos suspensivos (...) como elemento final de la lista o como elemento final de una sección delimitada de la lista indica que hay elementos adicionales en el modelo que satisfacen los criterios de selección pero no se muestran en la lista. En una visualización diferente de la lista, podrían aparecer estos elementos.

Estereotipo. Se puede aplicar un estereotipo a un elemento de la lista. La cadena del elemento va precedida por una palabra reservada encerrada entre comillas («»).

Cadena de propiedades. Una cadena de propiedades puede especificar una lista de propiedades de un elemento. Detrás del elemento va una lista separada por comas formada por las propiedades o restricciones, encerradas todas entre llaves ({ }).

Propiedades de grupo. Los sistemas y demás propiedades pueden aplicarse también a grupos de elementos de la lista. Si un estereotipo, una palabra reservada, una cadena de propiedades o una restricción aparece en solitario en una línea, entonces la línea no representa un elemento de lista. Lo que sucede es que las restricciones se aplican a los sucesivos elementos de la lista tal como si se hubieran situado directamente en todas las líneas. Este valor por defecto sigue siendo aplicable hasta que aparece en la lista otra propiedad de grupo. Se pueden cancelar todas las propiedades de grupo insertando una línea con una palabra reservada en blanco («») pero suele ser más claro poner todas las entradas que no estén sujetas a propiedades de grupo en la parte inicial de la lista. La Figura 13.131 muestra la aplicación de estereotipo a múltiples elementos de una lista.

Observe que las propiedades de grupo son meramente una forma cómoda de notación y que cada elemento del modelo posee su propio valor independiente para cada propiedad.

Nombre de compartimento. Los compartimentos pueden mostrar un nombre que indica de qué clase es el compartimento. El nombre se visualiza con una tipografía distintiva (tal como negrita en un tamaño más pequeño) centrado en la parte superior del compartimento. Esta capacidad resulta útil si se omiten algunos compartimentos o si se añaden compartimentos adicionales definidos por el usuario. Para una clase, los compartimentos predefinidos son los

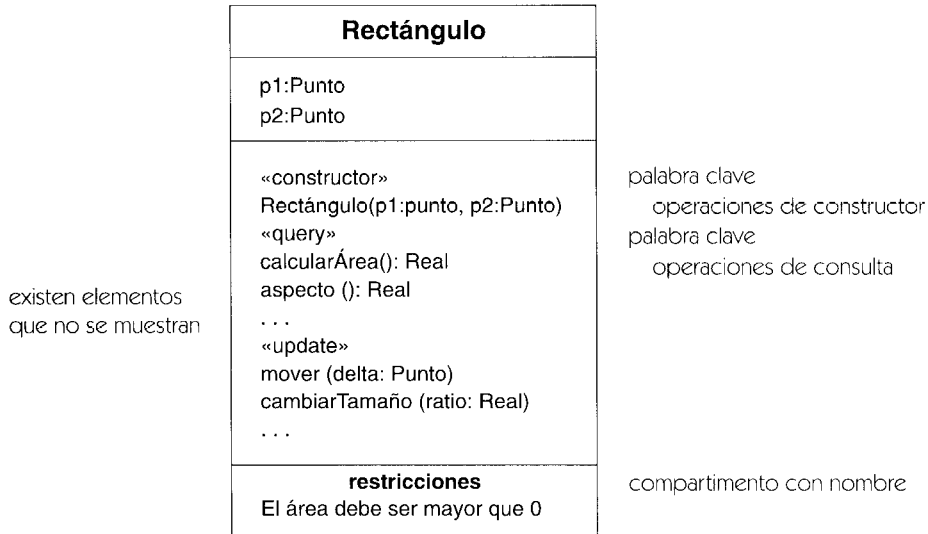


Figura 13.131 Una palabra reservada de estereotipo aplicada a grupos de elementos de una lista

atributos y operaciones con nombre. El compartimento de nombre de la clase debe estar siempre presente y por tanto no requiere ni admite un nombre de compartimento. La Figura 13.131 y la Figura 13.132 muestran compartimentos con nombre.

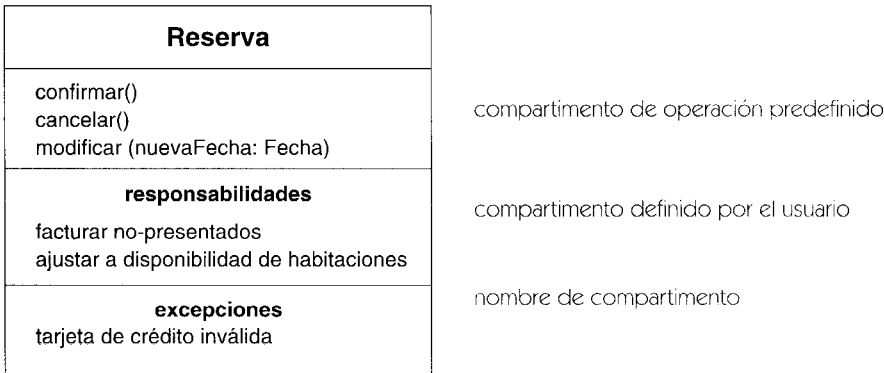


Figura 13.132 Compartimentos con nombre

Opciones de presentación

Orden. Una herramienta puede presentar los elementos de la lista colocándolos según su orden. En tal caso, el orden inherente de los elementos no es visible. Cada orden está basado en alguna propiedad interna y no indica información adicional del modelo. Entre las reglas de orden típicas se cuenta el orden alfabético, el orden por estereotipos (tales como constructores, destructores y por último métodos ordinarios), orden por visibilidad (pública, luego protegida, después privada) y así sucesivamente.

Filtrado. Los elementos de la lista pueden filtrarse empleando alguna regla de selección. La especificación de reglas de selección es una responsabilidad de la herramienta. Si una lista filtrada no muestra elementos, entonces no hay elementos que satisfagan el criterio y por tanto son invisibles. Es responsabilidad de la herramienta indicar o no de alguna forma la presencia de filtros locales o globales, aunque un diagrama independiente debería mostrar alguna indicación de ese filtrado si es que ha de resultar comprensible.

Si se suprime un compartimento, no se puede hacer deducción alguna respecto a la presencia o ausencia de sus elementos. Un compartimento vacío indica que no hay elementos que satisfagan el filtro de selección (si lo hubiere).

Observe que los atributos también se pueden mostrar por composición (véase la Figura 13.57).

lista de parámetros

Se trata de una especificación de los valores que recibe una operación o una plantilla. Una lista de parámetros es una lista ordenada de declaraciones de parámetros. La lista puede estar vacía, en cuyo caso la operación se invocará sin parámetros.

Véase parámetro.

Notación

Una lista de parámetros es una lista de declaraciones de parámetros separadas mediante comas y encerrada entre parámetros.

(parámetros_{lista,})

Los paréntesis se muestran aun cuando la lista esté vacía.

lista de propiedades

Alude a una sintaxis de texto adecuada para mostrar una propiedad o propiedades asociadas a un elemento, especialmente valores etiquetados, pero incluyendo también los atributos incorporados de los elementos del modelo.

Notación

Se trata de una o más especificaciones de propiedades separadas mediante comas y encerradas entre llaves. Las declaraciones de propiedad tienen la forma:

nombre-propiedad = valor

o bien

literal-propiedad

en donde en literal-propiedad es un valor enumerado exclusivo cuya aparición implica un nombre de propiedad único.

Ejemplo

```
{abstracto, autor=José, visibilidad=privada}
```

Opciones de presentación

Las herramientas pueden presentar las especificaciones de propiedades en líneas distintas o sin las llaves que las encierran, siempre y cuando estén marcadas adecuadamente para distinguir las de otras informaciones. Por ejemplo, las propiedades de una clase pueden estar enumeradas bajo el nombre de la clase y empleando algún tipo de letra distintivo, tal como cursiva o alguna otra tipografía. Éste es un problema de la herramienta.

Observe que las cadenas de propiedades se pueden utilizar para mostrar atributos incorporados, así como valores etiquetados, pero debería evitarse este tipo de utilización si la forma canónica es simple.

localización

Este concepto denota la ubicación física de una entidad propia del momento de la ejecución, tal como un objeto o un componente, dentro de un entorno distribuido. En UML, la localización es discreta y las unidades de localización son los nodos.

Véase también componente, nodo.

Semántica

El concepto de localización requiere el concepto de un espacio en que puedan existir las cosas. UML no modela toda la complejidad del universo tridimensional. Lo que hace es admitir un modelo topológico de espacios conectados mediante rutas de comunicación. Un nodo es un recurso de computación en el cual puede vivir una entidad propia del momento de la ejecución. Los nodos se conectan entre sí mediante rutas de comunicaciones que se modelan mediante asociaciones. La localización de una entidad se especifica haciendo referencia a un nodo. Dentro de un nodo, algunas entidades viven dentro de otras entidades anidadas. Por ejemplo, un objeto vive dentro de un componente o bien dentro de otro objeto. La localización de estas entidades es la entidad que las contiene.

Una instancia de objeto o de componente se puede trasladar a una nueva localización. Esto se puede modelar empleando la relación de conversión, que indica que en algún momento la primera entidad es sustituida por la segunda entidad, que posee otra localización diferente.

Notación

La localización de una instancia (incluyendo los objetos, instancias de componentes e instancias de nodos) dentro de otra instancia se puede mostrar físicamente por anidamiento, según se

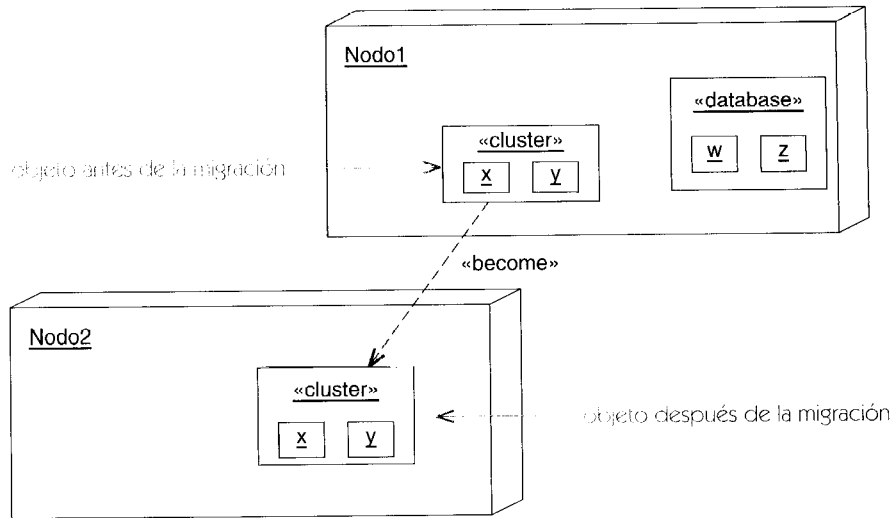


Figura 13.133 Nodos y migración de objetos

muestra en la Figura 13.133. La inclusión también se puede mostrar mediante flechas de composición. Alternativamente, una instancia puede poseer un marcador de propiedad llamado **localización** cuyo valor es el nombre de la instancia que lo contiene.

Si un objeto se traslada durante una interacción, puede aparecer como dos o más versiones con una transición de conversión **entre** versiones, tal como se muestra en la Figura 13.133. La flecha de conversión puede tener asociado un número de secuencia para mostrar el momento en que se traslada el objeto. Cada símbolo de objeto representa una versión del objeto durante una porción del tiempo total. Es preciso conectar los mensajes a la versión correcta del objeto (Figura 13.96).

llamada

Invocación a una operación.

Véase también activación, enviar, evento de llamada.

Semántica

Una llamada es la invocación de una operación en un punto determinado de la ejecución de un procedimiento. Mueve temporalmente un hilo de control desde el procedimiento que hace la llamada hacia el procedimiento llamado. La ejecución de un procedimiento que hace una llamada se bloquea durante la misma, ya que dicho procedimiento cede el control durante la ejecución de la operación y lo retoma cuando ésta termina. El procedimiento llamado recibe una lista de argumentos del procedimiento que realiza la llamada y un puntero implícito de retorno al procedimiento que realiza la llamada que apunta justo después de la instrucción de llamada. Cuando el procedimiento llamado finaliza, puede devolver una lista de valores.

A menudo la llamada se ejecuta dentro del espacio de direcciones del procedimiento que realiza la llamada, pero esto no es necesario para la semántica de la misma. Es más, resulta imposible hacerlo en un sistema distribuido, donde el receptor de la llamada puede estar separado físicamente del que llama. Mucho más importante resulta el hecho de establecer implícitamente el punto de retorno a la localización y entorno del procedimiento que realizó la llamada, pues permite restablecer el control al retornar de la misma. La localización del procedimiento que realiza una llamada se puede modelar mediante una línea de texto dentro de un procedimiento textual o mediante un estado dentro de una máquina de estados. Su entorno se modela utilizando una activación.

Una dependencia de uso de llamada modela una situación en la que una operación de la clase cliente (o la propia operación) llama a una operación de la clase proveedora (o a la propia operación). Esto se representa mediante el estereotipo «**call**».

Notación

Una llamada se representa en un diagrama de secuencia o de colaboración mediante un mensaje de texto dirigido a un objeto o clase destino.

Una dependencia de llamada se representa mediante una flecha discontinua etiquetada con el estereotipo «**call**», que sale de la clase que realiza la llamada y llega a la clase u operación llamada.

La mayoría de las llamadas se representarán como texto, formando parte del código de un procedimiento en un lenguaje de programación.

mal formado

Denota un modelo que se ha construido incorrectamente, por violar una o más de las reglas o restricciones predefinidas o especificadas por el modelo. Antónimo: bien formado.

Véase también conflicto, restricción.

Semántica

Un modelo que viola las reglas y restricciones de buena formación no es válido y por tanto tiene una semántica inconsistente. Si se intenta utilizar uno de estos modelos, se obtendrán resultados sin sentido. La herramienta de modelado tiene la responsabilidad de detectar los modelos mal formados e impedir su utilización en situaciones que pudieran dar lugar a problemas. Dado que el uso de ciertas estructuras extiende la semántica propia de UML, la verificación automática puede no ser posible en todos los casos. Además, no se puede esperar que las comprobaciones automáticas verifiquen la consistencia de las operaciones porque esto implicaría resolver el problema de la detención. Por tanto, las situaciones prácticas exigen una combinación de verificación automática y verificación humana.

Aun cuando un modelo terminado debe ser bien formado, las versiones intermedias del modelo pueden estar mal formadas en ocasiones, porque pueden ser fragmentos incompletos de al-

gún modelo final. La modificación de un modelo válido para producir otro modelo válido puede exigir ir pasando por modelos intermedios mal formados. Esto equivale a la modificación de programas de computador: el programa final que se le entrega al compilador tiene que ser válido, pero las copias de trabajo que residen en un editor de texto suelen no ser válidas. Por tanto, los modelos mal formados deben poder modificarse y almacenarse empleando las herramientas de apoyo.

máquina de estados

Especificación de las secuencias de estados por las que pasa un estado o una interacción en respuesta a los distintos eventos a lo largo de su vida, junto con las acciones de respuesta. Una máquina de estados se asocia a una clase, colaboración o método de origen y especifica el comportamiento de las instancias del elemento origen.

Véase también grafo de actividades, estado compuesto, evento, pseudoestado, estado, transición.

Semántica

Una máquina de estados es un grafo de estados y transiciones que describe la respuesta de un instancia de un clasificador frente a la recepción de eventos. Las máquinas de estados pueden estar asociadas a clasificadores, tales como las clases y los casos de uso, así como a colaboraciones y a métodos. El elemento al que está asociada la máquina de estados se denomina maestro de la máquina de estados.

Una máquina de estados completa es un estado compuesto que se ha descomprimido recursivamente para formar subestados. Los estados simples más internos no poseen subestados.

Semántica de ejecución de una máquina de estados

La semántica de la ejecución de una máquina de estados se discute en el resto de este artículo. Obsérvese que la sección siguiente describe los efectos semánticos de la ejecución de una máquina de estados y no debe tomarse como un enfoque de implementación. Hay muchas formas de implementar esta semántica, muchas de las cuales eliminan al compilar algunos de los pasos explícitos que se describen aquí. La mayor parte de esta semántica está descrita en otros artículos, pero se ha reunido aquí por comodidad.

En cualquier momento, uno o más estados pueden estar activados en la configuración de estado activa de la máquina de estados de un objeto o de otra instancia. Si un estado está activado, entonces se puede disparar una transición que abandone el estado, dando lugar a la ejecución de una acción y a la activación de otro estado o estados en lugar del estado original. Más de un estado hoja activo indica concurrencia interna. Existen restricciones para los estados que pueden estar activos concurrentemente, restricciones que vienen impuestas por la estructura de la máquina de estados y sus transiciones. En pocas palabras, si un estado secuencial compuesto está activado, entonces tiene que estar activo exactamente un subestado desjunto directo; si está activo un estado compuesto concurrente, todo subestado directo concurrente debe estar activo.

Disparo de transiciones y acciones

La suposición básica es que una máquina de estados procesa un evento de cada vez y termina con todas las consecuencias del evento antes de procesar otro. En otras palabras, los eventos no interaccionan con otros eventos durante el procesamiento de eventos. Esto se conoce con el nombre de procesamiento “ejecutar hasta finalizar”. No significa que toda la computación sea no interrumpible. Una computación extensa ordinaria se puede descomponer en una serie de pasos atómicos y la computación puede ser interrumpida por un evento externo entre pasos consecutivos. Esto está muy cerca de la situación física que se tiene dentro de un computador, en la cual las interrupciones pueden producirse en pasos discretos pero pequeños.

Una suposición derivada es que los eventos son asíncronos. Dos eventos no pueden suceder nunca exactamente al mismo tiempo, o más precisamente, si se producen dos eventos exactamente al mismo tiempo, se trata de una coincidencia y se pueden procesar como si se hubieran producido en cualquier orden, sin pérdida de generalidad. Los resultados de los diferentes órdenes de ejecución puede ser diferentes —las situaciones de competencia son una propiedad esencial de los sistemas concurrentes— pero no se puede suponer la simultaneidad en un mundo distribuido. Toda computación que haga esta suposición es incorrecta desde el punto de vista lógico y desde el punto de vista físico. En el mundo distribuido, la ejecución concurrente exige la independencia.

Conceptualmente, las acciones son instantáneas y los eventos nunca son simultáneos. En una implementación, la ejecución de acciones requiere un cierto tiempo pero lo importante es que las acciones son (conceptualmente) atómicas y no interrumpibles. Si un objeto recibe un evento mientras está ejecutando una acción, el evento se pone en cola hasta que finalice la ejecución. Los eventos sólo se manejan cuando no se está ejecutando ninguna acción. Si una acción envía una señal a otro objeto, entonces la recepción de la señal no es síncrona. Se maneja como cualquier otro evento, una vez finalizada la acción y la transición de la que forma parte. Una llamada a una operación suspende al emisor hasta que se haya ejecutado la operación. Si el receptor lo decide así, puede ser implementada como un método o como un evento de llamada que dispare la máquina de estados del receptor. Para evitar problemas en los largos períodos en que no se puedan procesar eventos, las acciones deberían ser breves. Las acciones no están destinadas a modelar regiones protegidas o largas computaciones interrumpibles, que deberían modelarse como submáquinas o como estados de actividad anidados. Esto permite el procesamiento de eventos y hace posible interrumpir las computaciones anidadas. Si se incluyen acciones largas en sistemas reales, los eventos no podrán procesarse oportunamente. Esto es la consecuencia de un mal modelo. Las acciones tienen que ser cortas en comparación con el tiempo de respuesta requerido por los eventos que puedan producirse.

Cuando un objeto no está llevando a cabo una acción, maneja inmediatamente cualquier evento que reciba. Conceptualmente, las acciones son instantáneas pero en la práctica requieren un cierto tiempo; por tanto, es preciso almacenar los nuevos eventos en una cola del objeto. Si no hay eventos en la cola, el objeto espera hasta que recibe un evento y entonces lo procesa. Conceptualmente, los objetos manejan un único evento de cada vez. Esto no es una limitación, porque se supone que los eventos son breves y atómicos. En una implementación real, los eventos se pueden poner en cola siguiendo un determinado orden. La semántica de UML, sin embargo, no especifica un orden para procesar eventos concurrentes y el creador del modelo tampoco deberá suponer que existe. Si es preciso procesar eventos en un determinado orden, entonces debe construirse una máquina de estados para imponer ese orden. Es probable que una implementación real impusiera alguna regla simple de orden.

En el momento en que un objeto maneja un evento, su configuración del estado activo puede contener uno o más estados concurrentes. Cada estado recibe una copia independiente del evento y actúa en consecuencia de ella independientemente. Las transiciones de los estados concurrentes se disparan independientemente. Un subestado puede cambiar sin afectar a los demás, salvo en el caso de alguna transición compleja, tal como una bifurcación o una unión (que se describirán más adelante).

Para cada estado activo de un objeto, las transiciones salientes del estado son candidatos al disparo. Una transición candidata se dispara si maneja un objeto cuyo tipo sea el disparador de la transición o bien algún descendiente suyo. La transición no se dispara si recibe un predecesor del evento. Cuando se maneja un evento y se dispara una transición, se evalúa la condición de guarda de la transición. Si el valor de la condición de guarda es verdadero, entonces se habilita la transición. La expresión booleana de la condición de guarda puede contener argumentos del evento disparador, así como atributos del objeto. Observe que las condiciones de guarda no pueden producir efectos secundarios. Esto es, no pueden alterar el estado del objeto ni del resto del sistema. Por tanto, el orden en que se evalúen será irrelevante para el resultado. Una condición de guarda sólo se evalúa cuando se maneja un evento. Si es falsa, no se vuelve a evaluar si alguna de sus variables cambia posteriormente de valor.

Para estructurar condiciones complejas, se puede modelar una transición con múltiples segmentos. El primer segmento tiene un evento disparador y va seguido por un árbol bifurcado de segmentos con condiciones de guarda. Los nodos intermedios del árbol son pseudoeestados, estados neutros que están presentes para estructurar las transiciones pero no pueden seguir activos al final de un paso que se ejecuta hasta finalizar. Toda posible ruta que pase a través del árbol de segmentos se considera una transición por separado y se puede seleccionar independientemente para su ejecución. Un segmento individual no se puede disparar por sí mismo. Todas las condiciones de guarda de una serie de segmentos tienen que ser ciertas, o bien la transición (incluyendo cualquiera de sus segmentos) no se disparará en modo alguno. En la práctica, las condiciones de guarda presentes en los puntos de bifurcación suelen particionar los posibles resultados. Por tanto, una implementación podría procesar la transición multisegmento paso a paso, pero no siempre.

Si no está habilitada ninguna transición, el evento se ignora sin más consideraciones. Esto no es un error. Si está habilitada exactamente una transición, se dispara. Si está habilitada más de una transición procedente de un único estado, entonces sólo se dispara una de ellas. Si no se ha especificado ninguna restricción, entonces la opción no es determinista. No se puede suponer que la opción será justa, predecible o aleatoria. Una implementación real podría proporcionar reglas para resolver conflictos, pero se advierte a los creadores de modelos que expliciten claramente su intención, en lugar de confiar en tales reglas. Tanto si se dispara alguna transición como si no, el evento se consume.

Las transiciones que abandonan un estado activo pueden ser seleccionadas para el disparo. Además, las transiciones de cualquier estado compuesto que contenga un estado activo son candidatas al disparo. Esto se puede considerar como una herencia de transiciones por parte de los estados anidados, de forma similar a la herencia de operaciones por parte de las subclases. Una transición desde un estado que contiene al actual sólo puede seleccionarse para el disparo si no se dispara ninguna transición de algún estado interno, en cuyo caso queda enmascarada por la transición interna.

Cuando se dispara una transición, se ejecuta cualquier acción que pudiera estar asociada a ella. Una expresión de acción puede hacer uso de los argumentos del evento que causa el dis-

paro, así como de los atributos del objeto poseedor o de valores que resulten alcanzables desde él. Las acciones son atómicas y finalizan antes de procesar eventos adicionales. Si una transición tiene múltiples segmentos, los parámetros del evento disparador están disponibles como evento actual implícito.

Si un objeto posee estados concurrentes, entonces no deben interactuar mediante memoria compartida. Los estados concurrentes están destinados a ser independientes y deberían basarse en conjuntos de valores diferentes. Todas las interacciones deben ser explícitas, mediante el envío de señales. Si dos estados concurrentes tienen que acceder a un recurso compartido, deberán enviar explícitamente señales al recurso, que puede entonces actuar como árbitro. Una implementación puede eliminar al compilar esta comunicación explícita, pero hay que tener cuidado para asegurarse de que no surjan conflictos peligrosos o sin sentido. Si las acciones concurrentes acceden realmente a valores compartidos, el resultado no es determinista.

Si se dispara una transición que cruza los límites de un estado compuesto, se pueden ejecutar acciones de entrada o de salida. Se puede producir el cruce de un límite porque el estado de origen y el estado destino de la propia transición se encuentran en diferentes estados compuestos. También puede suceder porque la transición que se dispara se ha heredado de un estado compuesto externo, forzando entonces al objeto a salir de uno o más estados internos. Observe que una transición interna no produce un cambio de estado y por tanto ni invoca acciones de entrada ni de salida.

Para determinar cuáles son las acciones de entrada y de salida que se ejecutan, busque el estado activo actual del objeto (que puede estar anidado dentro del estado compuesto que tenga la transición) y el estado destino de la transición. Busque entonces el estado compuesto más interno que contenga tanto al estado actual como al estado destino. Llamémosle *antepasado común*. Las acciones de salida del estado actual y de todos los estados que lo contengan hasta el antepasado común (exclusive) se ejecutarán en primer lugar. Después se ejecuta la acción de transición. Después de esto, se ejecutan las acciones de entrada de los estados contenedores hasta el antepasado común (exclusive), empezando por el estado más externo. En otras palabras, se sale de los estados uno por uno hasta llegar al antepasado común y después se va entrando en un estado tras otro hasta llegar al estado destino. Las acciones de entrada y salida del antepasado común no se ejecutan porque no ha cambiado. Este procedimiento asegura que todos los estados queden fuertemente encapsulados.

La acción de la transición se ejecuta después de haber ejecutado las posibles acciones de salida y antes haber ejecutado ninguna de las posibles acciones de entrada.

Observe que el disparo de una autotransición (una transición que va de un estado a sí mismo) dará lugar a la salida de todos los estados anidados dentro del estado de origen que puedan estar activos (la transición puede haber sido heredada de un estado compuesto que lo contenga). También da lugar a la ejecución de la acción de salida del estado de origen seguida por la ejecución de su acción de entrada. En otras palabras, se sale del estado y se vuelve a entrar en él. Si no se desea este efecto, entonces debería utilizarse en su lugar una transición interna del estado. Esto no dará lugar a un cambio de estado, aun cuando el estado activo esté anidado dentro del estado que posee la transición.

Durante la ejecución de un paso que se ejecuta hasta finalizar, todas las acciones tienen acceso a un evento actual implícito, que es el evento que ha disparado la primera transición de dicho paso. Dado que puede haber más de un evento que pudiera dar lugar a la ejecución de una acción, la acción puede discriminar según el tipo de evento actual (tal como en Ada o mediante una operación polimórfica) para ejecutar bifurcaciones de código alternativas.

Una vez efectuadas todas las acciones, el estado actual original está inactivo (salvo que se trate del estado destino), el estado destino de la transición está activo y es posible procesar eventos adicionales.

Se puede estructurar una transición con varios segmentos cuyos nodos intermedios sean estados de unión. Cada segmento puede poseer su propia acción. Las acciones pueden estar entrelazadas con acciones de entrada y de salida para la transición global. Con respecto a las acciones de entrada y de salida, toda acción de un segmento de transición se produce en el lugar en que ocurriría si el segmento fuera una transición completa. Véase la Figura 13.96 para observar un ejemplo.

Transiciones internas

Una transición interna tiene un estado de origen, pero no un estado destino. Su disparo no da lugar a un cambio de estado, aun cuando la transición que se dispara se haya heredado de un estado que lo contenga. Dado que no cambia el estado, no se ejecuta ninguna acción de entrada ni de salida. El único efecto que tiene una transición interna es la ejecución de su acción. Las condiciones para el disparo de una transición interna son exactamente las mismas que para una transición externa.

Observe que el disparo de una transición interna puede enmascarar a una transición externa que haga uso del mismo evento. Por tanto, puede tener sentido definir una transición interna que no posea una acción asociada. Según se ha indicado anteriormente, sólo se dispara una transición por evento dentro de cada región secuencial y las transiciones internas tienen prioridad con respecto a las transiciones externas.

Las transiciones internas son útiles para procesar eventos sin cambiar de estado.

Estados iniciales y finales

Para el encapsulamiento de estados, suele ser deseable separar el interior del estado de su exterior. También es deseable conectar las transiciones a un estado compuesto, sin conocer la estructura interna del estado. Esto se puede lograr empleando estados iniciales y finales dentro de un estado compuesto.

Un estado puede tener un estado inicial y un estado final. Un estado inicial es un pseudoestado —un estado neutro con la conectividad de los estados normales— y un objeto no puede permanecer en un estado inicial. Un objeto puede permanecer en un estado final, pero un estado final no puede tener ninguna transición que tenga un disparador explícito; su propósito es invocar a una transición de finalización de algún estado contenedor. Un estado inicial tiene que tener alguna transición de finalización saliente. Si hay más de una transición saliente, entonces todas ellas deben carecer de disparadores y sus condiciones de guarda deben particionar los posibles valores. En otras palabras, cuando se invoca el estado inicial tiene que dispararse exactamente una transición saliente. Un objeto no puede permanecer nunca en el estado inicial, por lo que efectuará inmediatamente una transición a un estado normal.

Si un estado compuesto posee un estado inicial, entonces las transiciones se pueden conectar directamente al estado compuesto como destino. Toda transición al estado compuesto es, im-

plícitamente, una transición al estado inicial situado dentro del estado compuesto. Si un estado compuesto carece de estado inicial, entonces no se pueden hacer transiciones que tengan como destino el estado compuesto; es preciso conectarlas directamente a los subestados. Un estado que tenga un estado inicial también puede tener transiciones conectadas directamente a los estados interiores, así como al estado compuesto.

Si un estado compuesto posee un estado final, entonces puede ser el origen de una o más transiciones salientes de finalización, esto es, atributos que carecen de eventos disparadores explícitos.

Una transición de finalización es, en realidad, una transición que está habilitada implícitamente por la terminación de la actividad dentro de su estado. Por tanto una transición a un estado final es una indicación de que la ejecución del estado compuesto ha finalizado. Cuando un objeto hace una transición a un estado final, las transiciones de finalización que salen del estado compuesto que lo contiene están preparadas para dispararse si se satisfacen sus condiciones de guarda.

Un estado compuesto también puede poseer transiciones salientes etiquetadas —esto es, transiciones con eventos disparadores explícitos—. Si se produce un evento que da lugar a que se dispare una de estas transiciones, entonces toda actividad en curso dentro del estado (para cualquier nivel de anidamiento) se cancelará, se ejecutarán las acciones de salida de los estados anidados finalizados y se procesará la transición. Estas transiciones suelen utilizarse para modelar excepciones y situaciones de error.

Transiciones complejas

Una transición que llega a un estado compuesto concurrente implica una transición a todos sus subestados concurrentes. Esto puede suceder de dos formas.

Una transición puede tener múltiples estados destino, uno dentro de cada subestado concurrente. Observe que esta transición con bifurcación sigue teniendo un único evento disparador, una condición de guarda y una sola acción. Se trata de una transición explícita a un estado compuesto que especifica directamente todos los destinos. Esto representa una bifurcación explícita del control para descomponerse en subhilos concurrentes.

Alternativamente, una transición puede omitir los destinos de uno o más subestados concurrentes, o bien puede poseer como destino el estado compuesto en sí. En tal caso, cada subestado concurrente omitido debe tener dentro un estado inicial para indicar su estado de inicio por defecto. En caso contrario, la máquina de estados está mal formada. Si se dispara la transición compleja, los subestados concurrentes destino explícitos se activan, tal como sucede con los estados iniciales de los demás subestados concurrentes. En pocas palabras, toda transición a un subestado concurrente implica una transición a los estados iniciales de los demás subestados concurrentes equivalentes, que no se han mencionado explícitamente. Una transición al estado compuesto en sí implica una transición a los estados iniciales de cada una de sus regiones concurrentes. Si un estado compuesto concurrente está activo, también están activas todas sus subregiones.

De forma similar una transición desde cualquier subestado concurrente implica una transición procedente de todos ellos. Si un evento da lugar a que se dispare una de estas transiciones,

se hará que concluya la actividad de los subestados restantes, se ejecutarán sus acciones de salida, se ejecutará la acción de la transición y el estado destino pasará a activado, reduciéndose así el número de estados activos concurrentes.

La transición al estado final de un subestado concurrente no implica la terminación de los demás subestados concurrentes (no se produce una transición que salga del subestado). Cuando todos los subestados concurrentes han llegado a sus estados finales, entonces se entiende que el estado compuesto que los contiene ha finalizado su actividad y se habilitan para el disparo todas las transiciones de finalización que salgan del estado compuesto.

Una transición compleja puede tener múltiples estados de origen y múltiples estados destino. En tal caso, su comportamiento es la combinación de la bifurcación y la unión que se han descrito anteriormente.

Estado de historia

Un estado compuesto puede contener un estado de historia, que es un pseudoestado. Si una transición heredada da lugar a una salida automática del estado compuesto, el estado “recuerda” el subestado que estuviera activo cuando se produjo la salida forzada. Una transición al pseudoestado de historia contenido en el estado compuesto indica que debe reestablecerse el subestado que se recuerda. Una transición explícita a otro estado o al estado que lo contiene no habilita el mecanismo de historia y se aplican las reglas habituales de las transiciones. Sin embargo, el estado inicial del estado compuesto puede estar conectado al estado de historia. En tal caso, una transición al estado compuesto invoca (indirectamente) al mecanismo de historia. El estado de historia sólo puede tener una transición saliente de finalización sin condición de guarda; el destino de esta transición es el estado de historia por defecto. Si no se ha entrado nunca en la región del estado o si se ha efectuado una salida normal, entonces una transición al estado de historia va al estado de historia por defecto.

Existen dos clases de estado de historia: un *estado de historia próxima* y un *estado de historia profunda*. El estado de historia próxima restaura los estados contenidos directamente (con profundidad uno) en el mismo estado compuesto que el estado de historia. El estado de historia profunda restaura el estado o estados que estuvieran activos antes de la última transición explícita que haya dado lugar a una salida del estado compuesto que la contiene. Puede contener estados anidados dentro del estado compuesto hasta cualquier profundidad. Un estado compuesto puede tener, como máximo, un estado de historia de cada clase. Cada uno puede poseer su propio estado de historia por defecto.

Debería evitarse el mecanismo de historia si se puede modelar directamente la situación, porque es complicado y no tiene necesariamente un buen acuerdo con los mecanismos de implementación. El mecanismo de historia profunda resulta especialmente problemático y debería evitarse, a favor de otros mecanismos más explícitos (y más implementables).

Notación

Un diagrama de estados muestra una máquina de estados o una parte anidada de una máquina de estados. Los estados se representan mediante símbolos de estado y las transiciones se re-

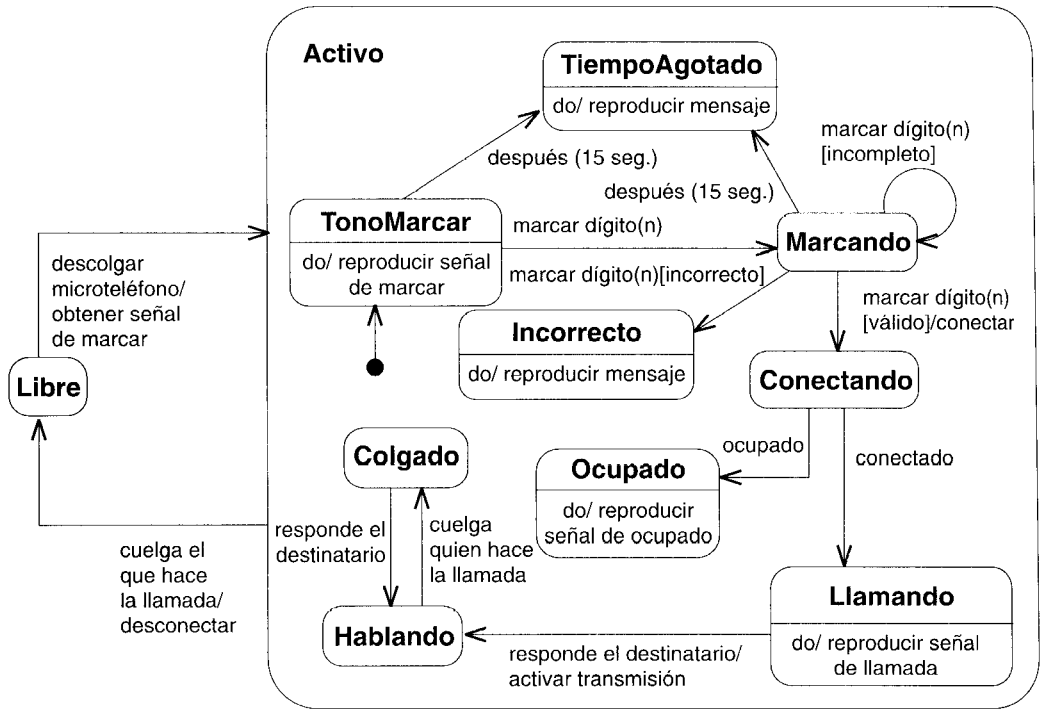


Figura 13.134 Diagrama de estados

presentan mediante flechas que conectan entre sí los símbolos de la seleccionada. Los estados también pueden contener subdiagramas, tanto físicamente como por apilamiento. La Figura 13.134 muestra un ejemplo.

La notación de diagramas de estado fue inventada por David Harel e incorpora ciertos aspectos de las máquinas de Moor (acciones de entrada) y de las máquinas de Mealy (acciones de transiciones) además de añadir los conceptos de estado anidado y de estado concurrente.

Para más detalles, véase estado, estado compuesto, submáquina, pseudoestado, acción de entrada, acción de salida, transición, transición interna y actividad. Para una forma variante de notación adecuada para el flujo de actividad, véase grafo de actividades. Véase también iconos de control para observar algunos símbolos opcionales destinados a ser utilizados en los diagramas de actividades, pero que pueden utilizarse en los diagramas de estado.

Discusión

Las máquinas de estados se pueden utilizar de dos maneras. Por tanto, su significado se puede comprender en cualquiera de estos dos sentidos. En un caso, la máquina de estados puede especificar el comportamiento ejecutable de su elemento maestro —típicamente, una clase—. En tal caso, la máquina de estados describe la respuesta del maestro cuando éste recibe eventos procedentes del resto del universo. La respuesta se describe mediante transiciones, cada una de las cuales indica lo que sucede cuando el maestro recibe un determinado evento mientras se encuentra en un estado dado. El efecto se expresa como una acción y un cambio de estado.

Entre las acciones se incluye enviar señales a otros objetos, que disparan las máquinas de estados de estos últimos. Las máquinas de estados proporcionan una especificación reduccionista del comportamiento de un sistema.

En el segundo caso, la máquina de estados puede utilizarse como una especificación de protocolo, que muestra el orden válido en que se pueden invocar las operaciones de una clase o de una interfaz.

En una de estas máquinas de estados, las transiciones que se disparan invocan a la operación deseada. Esto significa que se permite que el emisor invoque a la operación en ese momento. El protocolo de la máquina de estados no incluye acciones para especificar el comportamiento de la operación en sí. Muestra qué operaciones se pueden invocar en un orden particular. Esta máquina de estados especifica secuencias válidas de operaciones. Se está utilizando la máquina de estados como generador de secuencias en algún lenguaje (procedente de la teoría de lenguajes de las Ciencias de la Computación). Esta máquina hace las veces de una restricción aplicada al diseño del sistema. No es ejecutable directamente e indica lo que sucede si se produce una secuencia incorrecta, porque se supone que no pueden producirse. Es responsabilidad del diseñador del sistema asegurar que sólo se produzcan secuencias válidas. Este segundo modo de utilización es más abstracto que el primero, que especifica de forma ejecutable lo que sucede en todos los casos, pero suele ser cómodo, sobre todo en niveles altos y con codificación procedimental.

marca de tiempo

Notación que indica el instante en que se produce un evento o mensaje. Se pueden utilizar las marcas de tiempo en las restricciones.

Semántica

Una marca de tiempo se forma como una expresión que parte del nombre del mensaje. Se le puede dar nombre a un mensaje de una interacción para que sea posible formar expresiones de marca de tiempo a partir de él. En las expresiones siguientes mensaje es el nombre de un mensaje.

mensaje.horaEnvío() Instante en que se envía el mensaje

mensaje.horaRecepción() Instante en que se recibe el mensaje

Notación

Las expresiones de marca de tiempo se muestran en forma de texto.

Ejemplo

La restricción siguiente limita el tiempo necesario para generar una señal de marcar.

{tonomarcas.hora() - descolgado.hora() < 1 segundo}

marcador

Valor selector en una pareja de valor etiquetado. Representa el nombre de una propiedad definida en el tiempo de modelado.

marcador gráfico

Es un elemento de notación tal como la geometría, textura, trama de relleno, tipografía, color y demás.

Véase también uso de la tipografía.

Notación

Los símbolos de notación se construyen a partir de distintos marcadores gráficos. Ningún marcador gráfico tiene significado semántico por sí mismo, pero el objetivo de la notación es utilizar marcadores gráficos de forma tan consistente y ortogonal como sea posible.

Algunos marcadores gráficos se emplean para construir símbolos predefinidos de UML, mientras que otros marcadores gráficos se emplean en la notación canónica. Por ejemplo, no se ha asignado significado alguno a los colores, porque hay muchas impresoras que no lo producen y ciertas personas no distinguen todos los colores. Los marcadores gráficos que no estén reservados, como los colores, podrán emplearse dentro de las herramientas de edición para cualquier propósito propio de quien hace el modelado o de la herramienta.

UML permite una extensión gráfica limitada de su notación. Se puede asociar un icono o marcador gráfico (tal como una textura o un color) a cualquier estereotipo. UML no especifica la forma de la especificación gráfica. Sin embargo, existen muchos formatos de mapa de bits o de trazos que podrían ser empleados por un editor gráfico (aunque su transportabilidad es un problema difícil).

Se pueden concebir formas más generales de especificación y sustitución de iconos pero dejaremos estos problemas para el ingenio de los constructores de herramientas; advertimos que un uso excesivo de las capacidades de extensión puede dar lugar a dificultades para cambiar de herramientas usadas.

marco de trabajo

Una arquitectura genérica que proporciona una plantilla ampliable para su aplicación dentro de un dominio. Muy ampliamente difundido el término inglés “framework”.

Véase paquete.

materialización

Acto de materializar algo. *Véase* materializar.

materializar

Tratar como objeto algo que normalmente no se considera como objeto.

Discusión

La materialización tiene un significado filosófico y literario de larga trayectoria. Se utilizaba para describir la caracterización de conceptos abstractos como cosas o personas en la mitología y en la poesía. Por ejemplo, el dios Thor era una materialización del trueno. La teoría de los ideales de Platón invertía las cosas con respecto a los conceptos vigentes. Consideraba que los conceptos puros, como Belleza, Bien y Valor eran la realidad eterna; consideraba que las instanciaciones físicas eran copias imperfectas: es la materialización llevada hasta el último extremo.

La materialización es una de las ideas más útiles para la orientación a objetos y subyace en casi todos los aspectos del modelado. En primer lugar, la construcción de un modelo requiere imponer los objetos a un mundo continuo. Los seres humanos hacemos esto de forma natural en todas las frases que pronunciamos; un sustantivo es la materialización de una cosa y un verbo es la materialización de una acción. La materialización resulta especialmente útil cuando se aplica a las cosas de modelos o programas que no han comenzado siendo tratadas como objetos; tal podría ser el comportamiento dinámico. Casi todas las personas piensan en las operaciones como en objetos, pero ¿qué sucede con la ejecución (palabra que es, en sí, una materialización) de una operación? En general, las personas piensan que se trata de un proceso. Pero si se materializa y se le da un nombre (digamos activación) entonces resulta repentinamente posible darle propiedades, formar relaciones con otros objetos, manipularla y almacenarla. La materialización de un comportamiento transforma los procesos dinámicos en estructuras de datos que se pueden almacenar y manipular. Se trata de un concepto muy potente para el modelado y la programación.

mensaje

Denota el hecho de aportar información de un objeto (u otra instancia) a otro, con esperanzas de que esto dé lugar a alguna actividad. Un mensaje puede ser una señal o la llamada a una operación. La recepción de una instancia de mensaje se considera normalmente una instancia de evento.

Véase también llamada, colaboración, interacción, operación, enviar, señal.

Semántica

El término mensaje alude a enviar una señal de un objeto (el *emisor*) a otro u otros objetos (los *receptores*); también puede ser la llamada a una operación aplicada a un objeto (el *receptor*) por parte de otro objeto (el *emisor*, quien hace la llamada). La implementación de un mensaje puede adoptar distintas formas, tales como una llamada a procedimiento, una comunicación de procesos entre hilos activos, el envío explícito de un evento y así sucesivamente. En el nivel ló-

gico, enviar una señal y llamar a una operación son cosas similares. Ambas implican una comunicación procedente de un emisor y que llega a un receptor, en la cual se pasa información por valor que el receptor emplea para determinar lo que tiene que hacer. Una llamada puede considerarse una trama de señales que implica un envío con un argumento de puntero con retorno implícito que se emplea para enviar una señal de retorno al emisor. Se puede modelar una llamada como dos mensajes, un mensaje de llamada y otro mensaje posterior de retorno. En el nivel de implementación, las señales y las llamadas poseen propiedades y comportamientos detallados diferentes, por lo que se toman como elementos distintos en UML.

La recepción de una señal puede disparar una transición de la máquina de estados en el receptor. Una llamada puede manejarse de dos maneras, a elección del receptor (la elección del receptor tiene que efectuarse en el modelo). Se puede implementar una operación como cuerpo de un procedimiento (un método) que será invocado cuando llegue una llamada. Una vez finalizada la ejecución del procedimiento, el emisor recupera el control y puede recibir un valor opcional. Alternativamente, para un objeto activo, una llamada a una operación puede dar lugar a un evento de llamada, que a su vez dispara una transición de la máquina de estados. En este caso no existe el cuerpo del método, sino que la transición puede tener acciones. Cuando la transición ha finalizado, o bien inmediatamente si el evento de llamada no dispara una transición, el emisor recupera el control.

Los mensajes incluyen una expresión adecuada para un conjunto de objetos blanco. Se envía el mensaje a todos los objetos del conjunto. Salvo que se indique expresamente lo contrario (mediante una restricción), los mensajes se envían concurrentemente a todos los objetos del conjunto. Esto significa que el orden de ejecución es completamente arbitrario y podría ser paralelo. Si es preciso que se envíen los mensajes en un orden particular, deberán enviarse desde el interior de un bucle. Si se produce una llamada, el emisor recupera el control cuando han finalizado todas las llamadas.

El momento en que se envía o recibe un mensaje se puede representar mediante una expresión en el nombre del mensaje.

Véase marca de tiempo.

Estructura

El mensaje posee un emisor, un receptor y una acción.

Dentro de una interacción, el emisor es el rol de clasificador que envía el mensaje. El receptor es el rol de clasificador que recibe el mensaje. La acción es una llamada; una señal, una operación local aplicada al emisor o una acción primitiva tal como crear o destruir. La acción incluye una lista de argumentos, una expresión para un conjunto de receptores y una referencia de la operación o señal implicadas. También puede incluir una especificación de condicionalidad e iteración de la ejecución del mensaje.

Dentro de una interacción, los mensajes están relacionados mediante la relación predecesor-sucesor y la relación emisor-receptor. Esta última relación es aplicable a los métodos de procedimientos. Cada llamada añade un nivel de anidamiento a la secuencia. Dentro de una llamada, los mensajes se ordenan secuencialmente; existe la posibilidad de subsecuencias concurrentes.

La relación predecesor-sucesor (o de secuenciación) organiza los mensajes del hilo en una secuencia lineal. Si hay dos mensajes con un predecesor común y no están secuenciados de al-

guna otra forma, entonces se pueden ejecutar concurrentemente. Si un mensaje posee múltiples predecesores, tiene que esperar hasta que finalicen todos ellos. Ese mensaje es un punto de sincronización.

La relación emisor-receptor (o de activador) define una estructura de procedimientos anidada. El mensaje que llama a un procedimiento (empleando una acción de llamada) es el activador de todos los mensajes que forman el cuerpo del procedimiento invocado. Entre ellos, los mensajes invocados tienen una relación predecesor-sucesor que establece su orden relativo (y puede permitir la concurrencia).

Si un mensaje es una llamada, entonces se bloquea al emisor hasta que el procedimiento invocado finaliza y retorna. Sin embargo, si el receptor maneja la operación como evento de llamada entonces el retorno se produce cuando finaliza la transición inicial, tras lo cual el emisor recupera el control y el receptor puede reanudar su propia ejecución.

Las relaciones de secuenciación y de activador sólo relacionan a aquellos mensajes que posean la misma interacción.

Notación

La notación de los diagramas de secuencias y de los diagramas de colaboración es diferente.

Diagrama de secuencia

En un diagrama de secuencia, un mensaje se representa mediante una flecha continua que va desde línea de vida de un objeto (el emisor) hasta la línea de vida de otro objeto (el destino). Si la flecha es perpendicular a las líneas de vida, entonces se considera que la transmisión del mensaje es instantánea o al menos muy rápida en comparación con los mensajes externos. Si la flecha es oblicua, entonces se considera que la transmisión del mensaje tiene una duración, durante al cual se podrían enviar otros mensajes. En el caso de un mensaje que vaya del objeto hasta sí mismo, la flecha puede comenzar y acabar en la misma línea de vida. Las flechas de mensajes se organizan por orden secuencial verticalmente, empezando por arriba y avanzando hacia abajo. Si dos mensajes son concurrentes, su orden relativo no es significativo. Los mensajes pueden tener números de secuencia pero dado que el orden relativo de mensajes se muestra visualmente, los números de secuencia suelen omitirse.

Retardo de transmisión. Normalmente las flechas de mensajes se dibujan horizontalmente, lo cual indica que la duración necesaria para enviar el mensaje es atómica, esto es, que es breve en comparación con la granularidad que tiene la interacción y nada más puede “suceder” durante la transmisión del mensaje. Esta suposición es correcta en muchos computadores. Si el mensaje requiere un cierto tiempo para entregarlo, durante el cual puede ocurrir algo más (tal como un mensaje en dirección opuesta) entonces la flecha de mensaje se puede inclinar hacia abajo para que la punta de la flecha quede por debajo del origen de la misma.

Bifurcaciones. Las bifurcaciones se muestran mediante múltiples flechas que parten de un mismo punto y cada una de ellas está etiquetada con su propia condición de guarda. Dependiendo de si las condiciones de guarda son o no mutuamente excluyentes, esta estructura puede representar la condicionalidad o la concurrencia.

Iteración. Los conjuntos conectados de mensajes pueden estar encerrados y marcados como iteración. Los marcadores de iteración denotan que el conjunto de mensajes puede aparecer en múltiples ocasiones. Se puede especificar la condición de continuación para un procedimiento en la parte inferior de la iteración. Si hay concurrencia, entonces algunos mensajes del diagrama pueden formar parte de la iteración y otros pueden ejecutarse individualmente.

Diagramas de colaboración

En un diagrama de colaboración, los mensajes se representan mediante una flechita con etiqueta asociada a una ruta que une los objetos emisor y receptor. La ruta es la que se emplea para acceder al objeto blanco. La flecha apunta a lo largo de la ruta en la dirección del objeto destino. En el caso de que sea un mensaje que sale del objeto para volver a sí mismo, el mensaje aparece en una ruta que vuelve al mismo objeto y el extremo que llega al blanco posee la etiqueta «self».

Se puede asociar más de un mensaje a un enlace, tanto en la misma dirección como en direcciones distintas. El orden relativo de mensajes se muestra mediante la porción de número de secuencia que aparece en la etiqueta del mensaje.

Ambos diagramas

La flecha de mensaje tiene en su etiqueta el nombre del mensaje (el nombre de la señal o de la operación) y los valores de sus argumentos. La flecha también puede tener en su etiqueta un número de secuencia, para mostrar la posición del mensaje dentro de la interacción global. Los números de secuencia se pueden omitir en los diagramas de secuencias, en los cuales la situación física de la flecha muestra la posición relativa del mensaje, pero en los diagramas de colaboración resultan necesarios. Los números de secuencia resultan útiles en los dos tipos de diagramas para identificar los hilos de control concurrentes. También se puede poner en la etiqueta del mensaje una condición de guarda.

Tipo de flujo de control. Es posible emplear las siguientes variantes de puntas de flecha para mostrar las diferentes clases de flujo de control en los mensajes.

Punta de flecha con relleno



Llamada a procedimiento u otro flujo de control anidado. La secuencia anidada completa debe finalizar antes de reanudar la secuencia de nivel externo. Se puede emplear en llamadas normales a procedimiento. También se puede usar con objetos activos concurrentemente cuando uno de ellos envía una señal y espera a que finalice una secuencia de comportamiento anidada.

Punta de flecha sin relleno



Flujo de control plano. Cada flecha muestra la progresión al próximo paso de la secuencia. En el caso de procedimientos anidados, esto se corresponde con un barrido de las hojas del árbol de acciones efectuado lateralmente desde el fondo.

Media punta de flecha



Flujo de control asíncrono. Se emplea en lugar de la cabeza de flecha hueca para mostrar explícitamente un mensaje asíncrono entre dos objetos de una secuencia de procedimientos.

Flecha discontinua con punta sin relleno



Retorno de una llamada a procedimiento. La flecha de retorno puede suprimirse, por cuanto queda implícita al final de la activación.

Otras variantes

Se pueden mostrar otras clases de control, tales como “rehusar” o “tiempo agotado”, pero se tratan como extensiones del núcleo de UML.

Etiqueta de mensaje. La etiqueta tiene la sintaxis siguiente:

$$\text{predecesor}_{\text{opt}} \text{ condición-de-guarda}_{\text{opt}} \text{ expresión-secuencia}_{\text{opt}}$$

$$\text{lista-valores-retorno} :=_{\text{opt}} \text{ nombre-mensaje } (\text{argumentos}_{\text{lista}})$$

La etiqueta denota el mensaje enviado, sus argumentos y valores proporcionados y la situación del mensaje dentro de la interacción más extensa, incluyendo el anidamiento de llamadas, iteración, bifurcación, concurrencia y sincronización.

Predecesor. En una colaboración, el predecesor es una lista de números de secuencia separados por comas, y seguidos por una barra (/).

número-secuencia_{lista} /

La cláusula se omite si la lista está vacía.

Todo número-secuencia es una expresión-secuencia sin términos de recurrencia. Tiene que coincidir con el número-secuencia de algún otro mensaje.

El significado es que el flujo de mensajes no queda habilitado mientras no se hayan producido todos los flujos de mensaje cuyos números de secuencia estén enumerados (un hilo puede ir más allá del flujo de mensajes y la condición de guarda seguirá cumpliéndose). Por tanto, la condición de guarda representa una sincronización de hilos.

Observe que el mensaje que corresponde al número de secuencia precedente numéricamente es un predecesor implícito y no necesita ser enumerado explícitamente. Todos los números de secuencia que tienen igual prefijo forman una secuencia. El predecesor numérico es aquel en el cual el término final es uno menos. Esto es, el número 3.1.4.5 es el predecesor de 3.1.4.6.

En un diagrama de secuencia el orden visual determina la secuenciación y se muestra una sincronización por la presencia de múltiples mensajes que llegan a un mismo objeto antes de que el objeto envíe algún mensaje propio.

Expresión de secuencia. La expresión de secuencia es una lista de términos de secuencia separados por puntos y seguida por dos puntos (‘:’). Cada término representa un nivel de

anidamiento procedural dentro de la interacción global. Si es concurrente todo el control, entonces no se produce el anidamiento. Todos los términos de la secuencia poseen la siguiente sintaxis:

etiqueta $\text{recurrencia}_{\text{opt}}$

en donde etiqueta es entero o bien nombre

El entero representa el orden secuencial del mensaje dentro del nivel inmediatamente superior de llamadas a procedimientos. Los mensajes que difieren en un término entero están relacionados secuencialmente en ese nivel de anidamiento. Por ejemplo, el mensaje 3.1.4 sigue al mensaje 3.1.3 dentro de la activación 3.1.

El nombre representa un hilo de control concurrente. Los mensajes que difieren en el nombre final son concurrentes en ese nivel de anidamiento. Por ejemplo, el mensaje 3.1a y el mensaje 3.1b son concurrentes dentro de la activación 3.1. Todos los hilos de control son iguales dentro de cada profundidad de anidamiento.

La recurrencia representa la ejecución condicional o iterativa. Esto representa cero o más mensajes que se ejecutan, dependiendo de las condiciones. Las opciones son:

* [cláusula de iteración]	una iteración
[cláusula de condición]	una bifurcación

Una iteración representa una secuencia de mensajes en la profundidad de anidamiento dada. La cláusula de iteración se puede omitir (en cuyo caso las condiciones de iteración no estarán especificadas). La cláusula de iteración debería expresarse en pseudocódigo o en algún lenguaje de programación real; UML no prescribe su formato. Un ejemplo podría ser: $*[i:=1..n]$.

Una condición representa un mensaje cuya ejecución es contingente respecto a la veracidad de la cláusula de condición. La cláusula de condición debería expresarse en pseudocódigo o en algún lenguaje de programación real; UML no prescribe su formato. Un ejemplo podría ser: $[x > y]$.

Obsérvese que las bifurcaciones se anotan igual que una iteración sin asterisco. Pueden pensarse que se trata de una iteración limitada a un solo caso.

La notación de iteración supone que los mensajes de la iteración se ejecutarán secuencialmente. También existe la posibilidad de ejecutarlos concurrentemente. La notación empleada consiste en poner una doble línea vertical, indicadora de paralelismo, después del asterisco (*||).

Obsérvese que en una estructura de control anidada la recurrencia no se repite en niveles internos. Cada nivel de estructura especifica su propia iteración dentro del contexto que la contiene.

Signatura. Una signatura es una cadena que indica el nombre, argumentos y valor proporcionado por una operación, mensaje o señal. Poseen las propiedades siguientes:

lista-valores-proporcionados	Se trata de una lista de nombres separados mediante comas que denota los valores proporcionados por el mensaje en la
------------------------------	--

ejecución subsiguiente de la interacción global. Si el mensaje no proporciona ningún valor, entonces se omite el valor proporcionado y el operador de asignación.

nombre-mensaje

El nombre del evento surgido en el objeto destino (suele ser el evento que solicita la realización de una operación). Puede implementarse de varias maneras, *una* de las cuales es una llamada a una operación. Si se implementa mediante una llamada a procedimiento, entonces se trata del nombre de la operación y esa operación tiene que estar definida en la clase del receptor, o debe ser heredada por él. En otros casos puede ser el nombre de un evento que surge en el objeto receptor. En la práctica habitual, con sobrecarga de procedimientos, para identificar a una operación se necesita tanto el nombre del mensaje como la lista de tipos de los argumentos.

lista-argumentos

Se trata de una lista de argumentos separados mediante comas, encerrada entre paréntesis. Se pueden utilizar los paréntesis aun cuando la lista esté vacía. Cada argumento es una expresión en pseudocódigo o bien en un lenguaje de programación adecuado (UML no lo prescribe). La expresión puede utilizar los valores proporcionados por mensajes anteriores (del mismo alcance) y expresiones de navegación que partan del objeto original (esto es, atributos del mismo o enlaces que partan de él y rutas alcanzables desde ellos).

Ejemplo

Lo que sigue son ejemplos de la sintaxis de etiquetas de mensajes de control.

2: visualizar (x, y)	Mensaje simple
1.3.1: p:= buscar(especificaciones)	Llamada anidada que proporciona un valor
[x < 0]4: invertir (x, color)	Mensaje condicional
3.1 *: update ()	Iteración
A3, B4/ C2: copy(a,b)	Sincronización con otros hilos

Opciones de representación

En lugar de emplear expresiones de texto para los argumentos y valores proporcionados, se pueden mostrar símbolos de datos junto al mensaje (Figura 13.135). Un símbolo es un circulito etiquetado con la expresión de argumento o con el nombre del valor proporcionado. Posee una flechita que apunta a lo largo del mensaje (para un argumento) o en dirección opuesta al mensaje (para un valor proporcionado). Los símbolos representan argumentos y valores proporcionados.

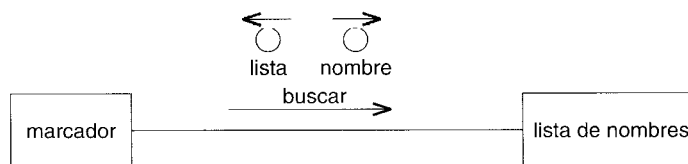


Figura 13.135 Símbolos de flujo de valores

La selección de sintaxis basada en texto o en símbolos es una opción de representación pero el texto resulta más compacto y se recomienda para la mayoría de los propósitos.

La sintaxis de los mensajes se puede expresar en la sintaxis de un lenguaje de programación tal como C++ o SmallTalk. Sin embargo, todas las expresiones de un mismo diagrama deberán utilizar una misma sintaxis.

metaclase

Una clase cuyas instancias son clases a su vez. Las metaclases se utilizan típicamente para construir metamodelos. Las metaclases se pueden modelar como un estereotipo de una clase empleando la palabra reservada «**metaclass**».

Véase también supratipo.

meta-metamodelo

Denota un modelo que define el lenguaje para expresar un metamodelo. La relación entre un meta-metamodelo y un metamodelo es análoga a la existente entre un metamodelo y un modelo. Este nivel de indirección sólo suele ser relevante para los constructores de herramientas, constructores de bases de datos y casos similares. UML está definido en términos de un meta-metamodelo, al que se denomina Servicio de Metaobjetos (MOF)⁴.

metamodelo

Este término denota un modelo que define el lenguaje empleado para expresar un modelo; también puede ser una instancia de un metamodelo. El metamodelo UML define la estructura de los modelos UML.

metaobjeto

Término genérico para todas las entidades que aparecen en un lenguaje de metamodelo. Por ejemplo, los metatipos, metaclases, metaatributos y metaasociaciones.

⁴ Meta-Object Facility en el original. *N. del T.*

metarelación

Este término agrupa aquellas relaciones que conectan a los descriptores con sus instancias. Entre ellas se cuentan la relación de instancia y la relación de supratipo.

método

Implementación de una operación. Especifica el algoritmo o procedimiento que da lugar a los resultados de una operación.

Véase también concreto, operación, realización.

Semántica

Un método es la implementación de una operación. Si una operación no es abstracta, entonces tiene que tener un método o un evento de llamada, bien sea definido en la clase que tiene esa operación o bien heredado de algún antecesor. Los métodos se especifican como expresiones de procedimientos, cadenas lingüísticas escritas en un determinado lenguaje (tal como C++, SmallTalk o algún lenguaje humano) que describen un algoritmo. El lenguaje debe ser adecuado para el propósito, por supuesto. Un lenguaje humano, por ejemplo, puede ser adecuado para un análisis inicial pero inadecuado para la generación de código.

La declaración de una operación implica la presencia de un método salvo que se declare abstracta el parámetro. En una jerarquía de generalización, cada declaración repetida de la operación implica un nuevo método que anula a cualquier método heredado de esa misma operación. Dos declaraciones representan a una misma operación si coinciden sus signatures.

Observe que un método es un procedimiento ejecutable —un algoritmo— y no simplemente una especificación de resultados. Una especificación de antes y después no es un método. Un método es un compromiso con la implementación y con los problemas de direcciones de un algoritmo, complejidad computacional y encapsulamiento.

En ciertos sentidos, un método puede tener propiedades más estrictas que las de su operación. Un método puede ser una consulta aun cuando la operación no se declare como consulta. Pero si la operación es una consulta entonces el método tiene que ser una consulta. Análogamente, un método puede fortalecer la propiedad de concurrencia. Una operación secuencial se puede implementar con un método con guarda o concurrente. En tales casos el método es consistente con las declaraciones de su operación. Sólo fortalece las restricciones.

Notación

La presencia de un método se indica mediante una declaración de operación que carece de la propiedad abstracta (Figura 13.136). Si la operación se hereda, entonces se puede mostrar el método repitiendo la declaración de la operación en texto normal (sin cursiva) para mostrar una operación concreta. El texto del cuerpo del método se puede mostrar en forma de una nota asociada a la entrada de lista de la operación, pero normalmente los cuerpos de los métodos no se

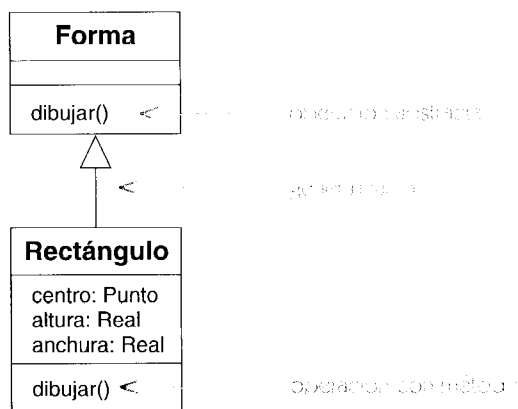


Figura 13.136 Un método de una operación no abstracta

muestran en todos los diagramas. Permanecen ocultos para que un editor de textos los muestre cuando se le pida.

miembro

Se trata del nombre de un constituyente estructural heredable y dotado de nombre que pertenece a un clasificador, bien sea un atributo, una operación o un método. Todo clasificador puede poseer una lista de cero o más de cada una de las clases de miembros. Las listas de miembros de una clase dada se denotan mediante una lista de cadenas situada dentro de un compartimento del símbolo clasificador.

Véase también lista.

modelo

Es una abstracción semánticamente completa de un sistema.

Véase también paquete, subsistema.

Semántica

Un modelo es una abstracción más o menos completa de un sistema desde un determinado punto de vista. Es completo en el sentido de que describe en su totalidad al sistema o entidad, con el nivel seleccionado de precisión y punto de vista. Los distintos modelos proporcionan, en esencia, puntos de vista independientes que se pueden manipular por separado.

Un modelo puede tener una jerarquía de contención formada por paquetes en la cual el paquete del nivel más elevado corresponde a todo el sistema. El contenido de un modelo es el cierre transitivo de sus relaciones de contención (propiedad) desde los paquetes de nivel superior hasta los elementos del modelo.

Los modelos también pueden incluir partes relevantes del entorno del sistema, representado, por ejemplo, por actores y por sus interfaces. En particular, se puede modelar la relación del entorno con los elementos del sistema. El sistema y su entorno forman un sistema más amplio y en un nivel de alcance más elevado. Por tanto, es posible relacionar elementos de diferentes niveles de forma sencilla.

Los elementos de diferentes modelos no se afectan directamente entre sí, pero suelen representar los mismos conceptos en distintos niveles de detalle o diferentes fases de desarrollo. Por tanto, las relaciones existentes entre ellos, tales como el seguimiento y el refinamiento, son importantes para el proceso de desarrollo en sí, y suelen capturar importantes decisiones de diseño.

Notación

Se puede mostrar un modelo como un paquete con el estereotipo «**model**». Sin embargo, poco es el detalle de notación que se puede mostrar acerca de los modelos. Las herramientas pueden mostrar listas de modelos pero los modelos tienen pocas relaciones entre sí. Lo más útil es la capacidad de pasar desde el nombre de un modelo hacia su paquete de más alto nivel o hasta un mapa de su contenido global.

Discusión

No hay ninguna vista de un sistema que pueda ser completa per se y lo mismo puede decirse de los sistemas. Siempre hay conexiones con el mundo exterior y los modelos nunca llegan a los límites de la realidad. Por tanto, el concepto de modelo cerrado siempre es una aproximación en la cual es preciso especificar unos límites arbitrarios para trabajar en la práctica.

Los modelos UML se representan como una jerarquía de paquetes que hace hincapié en una visualización del sistema. Cada modelo puede poseer su propia jerarquía de niveles y esa jerarquía puede ser similar o distinta de la jerarquía de niveles de otras vistas del sistema.

modelo de casos de uso

Modelo que describe los requisitos funcionales de un sistema u otro clasificador en términos de casos de uso.

Véase actor, caso de uso.

Semántica

El modelo de casos de uso representa la funcionalidad de un sistema u otro clasificador tal como lo perciben quienes interactúan con el sistema desde el exterior. Los modelos de casos de uso se muestran en un diagrama de casos de uso.

módulo

Término que denota una unidad para el almacenamiento y manipulación del software. Entre los módulos se cuentan los módulos de código fuente, los módulos de código binario y los módulos de código ejecutable. La palabra no corresponde a una única estructura de UML, sino que incluye varias estructuras.

Véase componente, paquete, subsistema.

muchos

Se trata de una abreviatura de la multiplicidad **0..*** —esto es, cero o más sin límites—. En otras palabras, un tamaño carente de restricciones.

Véase multiplicidad.

multiobjeto

Alude a un rol de clasificador que denota un conjunto de objetos y no un único objeto.

Véase también rol de clasificador, colaboración, mensaje.

Semántica

El multiobjeto es un rol de clasificador que denota un conjunto de objetos, normalmente el conjunto de objetos del extremo *muchos* de una asociación. Se utiliza un multiobjeto dentro una colaboración para mostrar operaciones que tratan a todo el conjunto de objetos como a una unidad y no a un único objeto. Por ejemplo, una operación para buscar un objeto dentro de un conjunto opera sobre todo el conjunto y no sobre un objeto individual. El modelo estático subyacente no queda afectado por este agrupamiento.

Notación

Los multiobjetos se representan mediante dos rectángulos en los cuales el rectángulo superior está desplazado vertical y horizontalmente para sugerir una pila de rectángulos (Figura 13.137).

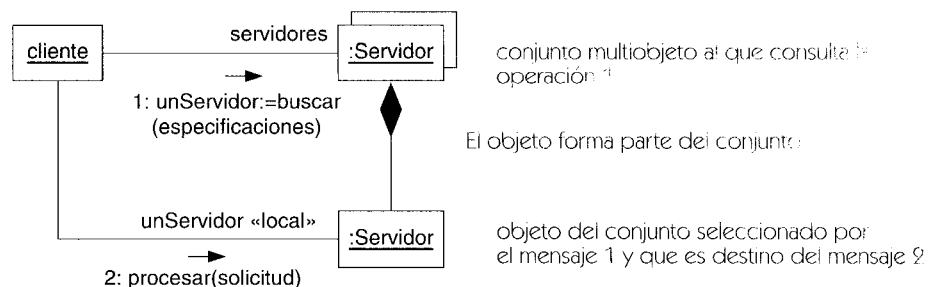


Figura 13.137 Multiobjeto

Una flecha de mensaje situada junto al símbolo del multiobjeto denota un mensaje que llega al conjunto de objetos —por ejemplo, una operación de selección para buscar un objeto individual.

Se necesitan dos mensajes para aplicar una operación a todos los objetos de un conjunto de objetos asociado: una iteración del multiobjeto para extraer enlaces de los objetos individuales y después un mensaje que se envía a cada uno de los objetos, empleando el enlace (temporal). Esto se puede omitir en un diagrama combinando los mensajes en uno que incluya una iteración y una aplicación a cada objeto. El nombre de rol destino recibe un indicador de *muchos* (*) para mostrar que están implicados muchos enlaces. Aun cuando esto puede escribirse como un único mensaje, en el modelo subyacente (y en todo código real) se necesitarán las dos capas de estructura (iteración para hallar los enlaces y un mensaje que hace uso de cada enlace) mencionadas anteriormente.

Un objeto del conjunto se representa mediante un símbolo normal de objeto pero puede estar asociado al símbolo de multiobjeto empleando un enlace de composición para indicar que forma parte del conjunto. Una flecha de mensaje que va hacia el símbolo del objeto simple denota un mensaje a un objeto individual.

Típicamente, un mensaje de selección enviado a un multiobjeto proporciona una referencia de un objeto individual al cual enviará entonces un mensaje el emisor original.

multiplicidad

Una especificación del intervalo de valores de cardinalidad admisibles —el tamaño, esto es— que puede adoptar un conjunto. Se pueden dar especificaciones de multiplicidad para extremos de asociación, partes de clases compuestas, repeticiones de mensajes y otros propósitos. En esencia una multiplicidad es un subconjunto (posiblemente infinito) de los enteros no negativos. Contrastar con: cardinalidad.

Véase también multiplicidad (de asociación), multiplicidad (de clase).

Semántica

La multiplicidad es una restricción de la cardinalidad (tamaño) de un conjunto. En principio, se trata de un subconjunto de los enteros no negativos. En la práctica, suele tratarse de un conjunto finito de intervalos enteros; en casi todas las ocasiones se trata de un único intervalo con un valor mínimo y otro máximo. Todo conjunto debe ser finito, pero el límite superior puede ser finito o ilimitado (una multiplicidad ilimitada se denomina “muchos”). El límite superior tiene que ser mayor que cero; en todo caso, una multiplicidad que sólo fuera cero no sería demasiado útil porque sólo permitiría el conjunto vacío). La multiplicidad se codifica como una cadena.

En la mayoría de los casos se puede especificar una multiplicidad como un intervalo entero —una cardinalidad máxima y mínima—pero en general puede ser un subconjunto discontinuo de los enteros no negativos. El conjunto de enteros puede ser infinito —esto es, el límite superior puede ser ilimitado (pero obsérvese que toda cardinalidad concreta del conjunto es finita).

Para casi todos los propósitos prácticos, este conjunto de enteros se puede especificar como una lista de intervalos enteros disjuntos e inconexos. Un intervalo es un conjunto de

enteros contiguos caracterizado por su valores mínimo y máximo. Algunos conjuntos infinitos no pueden especificarse de este modo —por ejemplo, el conjunto de los enteros pares— pero suele perderse poco por incluir simplemente los saltos. Para la mayoría de los propósitos de diseño, un único intervalo con un valor mínimo y máximo basta para toda la especificación de multiplicidad, porque un propósito esencial de la multiplicidad es limitar la cantidad de espacio de almacenamiento que pudiera ser necesaria.

Véase multiplicidad (de asociación) y multiplicidad (de clase) para obtener detalles específicos de la utilización de la multiplicidad con estos elementos.

Notación

La multiplicidad suele especificarse mediante una expresión de texto formada por una lista de intervalos enteros separados mediante comas, cada uno de los cuales tiene la forma

`minimo..maximo`

en donde `minimo` y `maximo` son enteros o bien `maximo` puede ser un “*” que denota un límite superior no acotado. Una expresión tal como `2..*` se lee en la forma “2 o más”.

Los intervalos también pueden tener la forma:

`número`

en donde `numero` es un entero que representa un intervalo de un único tamaño.

La expresión de multiplicidad que consta de un único asterisco:

`*`

equivale a la expresión `0..*` —esto es, indica que la cardinalidad es ilimitada (“cero o más, sin límites”)—. Esta multiplicidad de frecuente aparición se lee en la forma “muchos”.

Ejemplo

`0..1`

`1`

`0..*`

`*`

`1..*`

`1..6`

`1..3,7..10,15,19..*`

Reglas de estilo

- Preferiblemente, los intervalos deberían crecer de forma monótona. Por ejemplo, **1..3,7,10** es preferible a **7,10,1..3**.
- Dos intervalos contiguos deberían combinarse en un único intervalo. Por ejemplo, **0..1** es preferible a **0,1**.

Discusión

Las expresiones de multiplicidad pueden contener variables pero tiene que resolverse como valores enteros cuando el modelo esté completo, esto es, deben ser parámetros o constantes. La multiplicidad no está destinada a ser evaluada dinámicamente en el ámbito del tiempo de ejecución, como los límites de una matriz dinámica. Tiene como objeto especificar el posible intervalo de valores (en el caso peor) que podría adoptar un conjunto y la aplicación debe adaptar en consecuencia sus estructuras de datos y sus operaciones. Se trata de una constante del tiempo de modelado. Si el límite es variable en tiempo de ejecución, entonces la elección correcta de multiplicidad es *muchos* (0..*).

La multiplicidad puede suprimirse en un diagrama pero sigue existiendo en el modelo subyacente. En un modelo ya terminado, no tiene sentido hablar de una multiplicidad “no especificada”. No saber la multiplicidad es lo mismo que decir que es muchos, porque en ausencia de todo conocimiento la cardinalidad podría tomar cualquier valor, que es justamente lo que significa muchos.

Véase valor no especificado.

multiplicidad (de asociación)

Expresa la multiplicidad especificada en un extremo de la asociación.

Véase multiplicidad.

Semántica

La multiplicidad asignada a un extremo de asociación declara el número de objetos que pueden satisfacer el puesto definido por ese extremo de asociación.

Para una asociación binaria, la multiplicidad del extremo destino limita el número de objetos de la clase blanco que se pueden asociar a un objeto simple dado del otro extremo (el origen). Típicamente la multiplicidad se da como un intervalo de enteros (Véase multiplicidad para una definición más general.) Entre las multiplicidades se cuentan exactamente uno; cero o uno, cero o más sin límites y uno o más sin límites. La frase “cero o más sin límites” suele llamarse muchos.

En una asociación n -aria, la multiplicidad se define con respecto los $n-1$ extremos restantes. Por ejemplo, dada una asociación ternaria entre clases (A, B, C), la multiplicidad del extremo C indica el número de objetos C que pueden aparecer en asociación con una pareja particular de objetos A y B. Si la multiplicidad de esta asociación es (muchos, muchos, uno), entonces para cada posible pareja (A, B) existe un único valor de C. Sin embargo, para una pareja (B, C) dada pueden existir muchos valores de A y es posible que haya muchos valores de A, B y C que participen en la asociación.

Véase asociación n -aria para una discusión de la multiplicidad n -aria.

Notación

La multiplicidad se muestra mediante una cadena de multiplicidad que se sitúa junto al extremo de la ruta a la cual se aplica (Figura 13.138). Un intervalo de números tiene la forma $n1 \dots n2$.

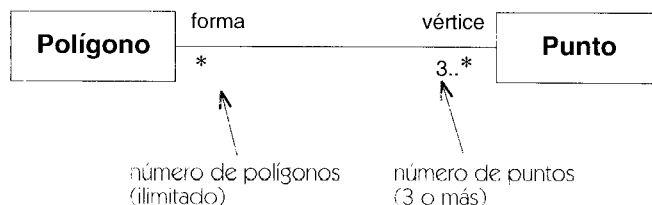


Figura 13.138 Multiplicidad de asociación

Véase multiplicidad para más detalles respecto a la sintaxis y formas más generales de especificarla (aunque posiblemente sean más generales de lo que resulta necesario en la práctica normal).

multiplicidad (de atributo)

Dícese del posible número de valores de un cierto atributo que puede haber en cada objeto.

Semántica

La multiplicidad asociada a un atributo declara el número de valores que se pueden almacenar en un objeto que tenga ese atributo.

La multiplicidad habitual es de exactamente uno (1..1), lo cual significa que todo objeto posee un único valor de ese atributo. Entre otras multiplicidades frecuentes se cuentan cero o uno (un valor opcional o “anulable”); cero o más, sin límites (un conjunto de valores) y uno o más sin límites (un conjunto no vacío de valores). La frase “cero o más sin límites” suele denominarse muchos.

Notación

La multiplicidad se representa mediante una cadena de multiplicidad entre corchetes situada junto al nombre del atributo y antes de los dos puntos (Figura 13.139). Si no hay corchetes, entonces la multiplicidad es exactamente uno (un valor escalar, el valor por defecto).

exactamente un nombre
cualquier número de teléfonos
entre 1 y 3 referencias

Cliente
nombre: Nombre teléfono [*]: Cadena referencias [1..3]: Cliente

Figura 13.139 Multiplicidad de atributos

multiplicidad (de clase)

Término que expresa el rango de posibles cardinalidades de las instancias de una clase, esto es, el número de instancias que pueden existir legítimamente en un mismo instante.

Semántica

Cuando se le aplica a una clase, la multiplicidad declara el número de instancias de la clase que pueden existir. El valor normal por defecto es ilimitado, pero hay ocasiones en que es útil una multiplicidad finita, especialmente para declarar clases unitarias, esto es, clases que sólo pueden tener una instancia y que normalmente se necesitan para establecer el contexto y los parámetros de todo el sistema.

La otra aplicación de la multiplicidad a las clases es en el interior de una colaboración, en la cual puede estar asociada a un rol de clasificador para especificar el número de instancias que se pueden asociar al rol en una instancia de la colaboración.

Notación

La multiplicidad de una clase o de un rol de clasificador se muestra mediante una cadena de multiplicidad situada en la esquina superior derecha del símbolo de rectángulo (Figura 13.140). Se puede omitir la cadena si la multiplicidad es muchos (ilimitada).



Figura 13.140 Multiplicidad de clase

navegabilidad

La navegabilidad indica si es o no posible recorrer una asociación binaria situada dentro de expresiones de una clase para obtener el objeto o conjunto de objetos asociado a una instancia de la clase. El concepto no es aplicable a las asociaciones *n*-arias (véase el texto). La propiedad de navegabilidad es una enumeración cuyos valores son **true** (navegable) y **false** (no navegable).

Véase también eficiencia de navegación.

Semántica

La navegabilidad indica si se puede o no emplear un nombre de rol en las expresiones para recorrer una asociación de un objeto a un objeto o conjunto de objetos de la clase asociada al extremo de la asociación que lleva ese nombre de rol. Si la navegabilidad es cierta, entonces la

asociación define un pseudoatributo de la clase que está en el extremo opuesto al nombre de rol —esto es, que se puede utilizar el nombre de rol en expresiones similares a un atributo de la clase para obtener valores—. El nombre de rol se puede emplear también para expresar restricciones.

La falta de navegabilidad implica que la clase situada al otro extremo del nombre de rol no es capaz de “ver” la asociación y por tanto no puede utilizarla para formar una expresión. Una asociación sin navegabilidad no crea una dependencia de la clase origen respecto a la clase destino, pero quizá se cree una dependencia por alguna otra causa.

La falta de navegabilidad no implica que no haya forma de recorrer la asociación. Si se puede recorrer la asociación en la otra dirección, quizá sea posible investigar todas las instancias de la otra clase para hallar aquellas que llevan a un objeto, invirtiendo de este modo la asociación. Este enfoque podría, incluso, llegar a ser práctico en casos pequeños.

La navegabilidad no está definida en asociaciones n -arias porque requeriría especificar conjuntos de clases a los cuales o desde los cuales habría que navegar. Esto podría hacerse pero resulta demasiado complicado para ser útil como propiedad básica. Eso no significa que no se puedan recorrer las asociaciones n -arias, sino simplemente que la especificación de su recorrido es complicada y no resulta adecuada para una sencilla definición booleana.

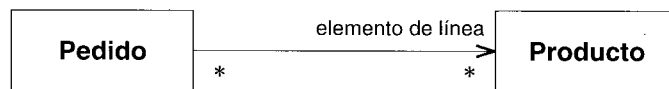
La navegación suele tener la connotación de eficiencia de navegación, aun cuando esto no es un requisito estricto de las reglas de UML.

Notación

Las asociaciones navegables se muestran con una punta de flecha situada en el extremo de la ruta de asociación asociada a la clase destino. La flecha indica la dirección de recorrido (Figura 13.141). El adorno de navegabilidad se puede suprimir (normalmente, en todas las asociaciones del diagrama). Las puntas de flecha pueden no estar asociadas a ningún extremo de las asociaciones, a un extremo o a los dos.

Para mayor comodidad, se pueden omitir las puntas de flecha en las asociaciones que son navegables en ambas direcciones. En teoría, esto se puede confundir con una asociación que no sea navegable en ninguna dirección, pero este tipo de asociación es poco probable en la práctica y se puede indicar explícitamente si ocurriera.

No hay necesidad de una notación para la navegabilidad “indecisa”. Si no se ha decidido la navegabilidad, entonces en el caso general será bidireccional. Toda decisión sobre la navegabilidad puede únicamente restringirla o dejar que sea completamente general.



No navegable:

Un producto no contiene
una lista de pedidos.

Navegable:

Todo pedido posee
una lista de productos.

Es posible hallar los pedidos de un producto, pero es necesario buscarlos fuera del producto.

Figura 13.141 Navegabilidad

navegable

Todas aquellas asociaciones o enlaces de una expresión que se pueden recorrer. Su propiedad de navegabilidad es **verdadera**. Estos enlaces suelen implementarse como un puntero o un conjunto de punteros.

Véase navegabilidad, eficiencia de navegación.

navegación

Acción de recorrer las conexiones de un grafo y especialmente de recorrer enlaces binarios y atributos de un modelo de objetos para hacer corresponder a un objeto con un valor. En este último caso, la ruta de navegación se puede expresar como una secuencia de nombres de atributos o de nombres de rol.

Véase navegabilidad.

nodo

El término nodo describe un objeto físico en tiempo de ejecución que representa un recurso computacional; generalmente tienen al menos memoria y con cierta frecuencia poseen capacidades de procesamiento. Los objetos de tiempo de ejecución y las instancias de componentes de tiempo de ejecución también pueden residir en nodos.

Véase también localización.

Semántica

Entre los nodos se cuentan los dispositivos de computación pero también (al menos en los modelos de negocios) los recursos humanos o los recursos de procesamiento mecánico. Los nodos se pueden representar como descriptores y como instancias. Todo nodo define una localización en la cual pueden residir instancias computacionales de tiempo de ejecución, tanto instancias de objetos como instancias de componentes.

Los nodos físicos poseen muchas propiedades adicionales, tales como capacidad, rendimiento y fiabilidad. UML no prescribe estas propiedades, porque existe un elevado número de posibilidades, pero es posible modelarlas en UML empleando estereotipos y valores etiquetados.

Los nodos pueden conectarse a través de asociación para mostrar rutas de comunicación. Es posible dar estereotipos a las asociaciones para distinguir los distintos tipos de rutas de comunicación o sus diferentes implementaciones.

Los nodos forman parte, inherentemente, la vista de implementación y no de la vista de análisis. En los modelos de despliegue aparecen normalmente instancias de nodos, más que tipos de nodos. Aun cuando los tipos de nodos pueden tener un significado, los tipos de los nodos individuales suelen permanecer en el anonimato.

Todo nodo, como clasificador que es, puede poseer atributos. En casi todas las ocasiones se muestran instancias de nodos en los diagramas de despliegue. Los descriptores de nodo tienen una utilidad más reducida.

Notación

Los nodos se representan mediante una figura que parece la proyección descentrada de un cubo.

Los descriptores de nodos tienen la sintaxis:

tipo-nodo

en donde tipo-nodo es en nombre de clasificador.

Las instancias de nodo tienen nombre y nombre de tipo. El nodo puede tener una cadena de nombre subrayada en su interior o por debajo. La cadena de nombre tiene la sintaxis:

nombre : tipo-nodo

El nombre es el nombre del nodo individual (si lo hubiere). El tipo-nodo dice qué clase de nodo es. Estos elementos son opcionales individual o conjuntamente.

Se pueden emplear flechas de dependencia (flechas discontinuas con la punta de flecha en el componente) para mostrar la capacidad de un nodo para admitir un determinado tipo de componente. Se puede emplear un estereotipo para indicar con precisión el tipo de dependencia.

Es posible que existan instancias de componentes y de objetos almacenadas en el interior de símbolos de nodo. Esto indica que los elementos residen en las instancias del nodo. También se puede mostrar la contención por medio de rutas de asociación de agregación y composición.

Es posible conectar unos nodos con otros mediante símbolos de asociación. Una asociación entre dos nodos indica una ruta de comunicación entre ellos. La asociación puede tener un estereotipo para indicar la naturaleza de la ruta de comunicación (por ejemplo, la clase de canal o de red).

Ejemplo

La Figura 13.142 muestra dos nodos que contienen un objeto (cluster) que va de un componente situado en un nodo a otro componente situado en el otro.

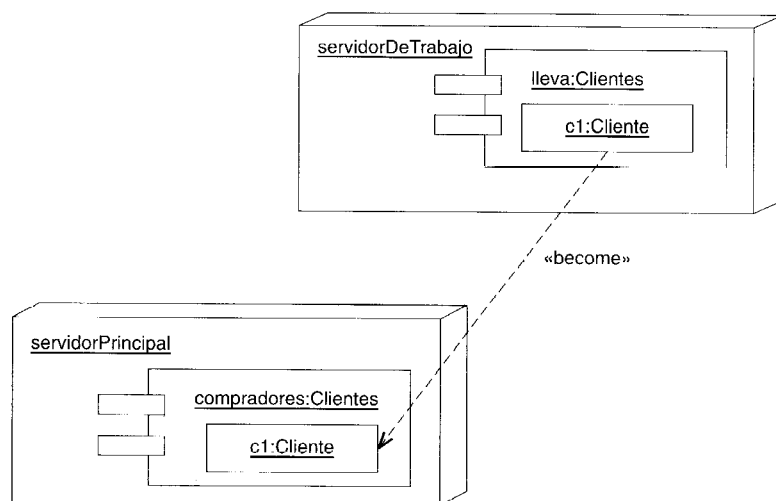


Figura 13.142 Migración entre nodos

no interpretado

Reserva de espacio para uno o más tipos cuya implementación no está especificada por el UML. Todo valor no interpretado posee una representación en forma de cadena. En cualquier implementación física, tal como un editor de modelos, la implementación tendría que ser completa, así que no habría valores no interpretados.

nombre

Término que denota la cadena que se emplea para identificar un elemento de un modelo.

Véase también espacio de nombres.

Semántica

Un nombre es un identificador —una secuencia de caracteres de algún alfabeto finito y predefinido—, en algún lenguaje definido. Una implementación puede imponer restricciones relativas a la forma de los nombres, tales como la exclusión de ciertos caracteres (por ejemplo, los signos de puntuación), restricciones relativas a los caracteres iniciales y demás. En particular, se supone que los nombres se pueden utilizar como selectores y clave de búsqueda en diferentes conjuntos de datos. Por ejemplo, los nombres del alfabeto romano suelen incluir letras mayúsculas y minúsculas; numerales y uno o más separadores tales como el guión inferior y el normal, mientras que otros signos de puntuación dependen de la implementación.

Tanto las herramientas como los lenguajes pueden imponer límites razonables sobre la longitud de las cadenas y el conjunto de caracteres que utilizarán para los nombres; es posible que sean más restrictivas que las correspondientes a las cadenas arbitrarias, tales como los comentarios.

Los nombres se definen en el interior de un espacio de nombres, tal como un paquete o una clase. Dentro del espacio de nombres, cada nombre tiene que ser único dentro de su propio grupo semántico, tal como el de los clasificadores, estados, atributos y demás; los nombres de grupos diferentes pueden coincidir (aunque esto debería evitarse para no inducir a confusión). Todo espacio de nombres, salvo el sistema completo, está contenido dentro de otro espacio de nombres. Los nombres de todos los espacios de nombres anidados y el nombre del elemento final se pueden componer en una sola cadena de nombre de ruta.

Notación

Los nombres se muestran como cadenas. Normalmente los nombres se muestran en una sola línea y contienen únicamente caracteres imprimibles. La notación canónica para los nombres incluye los caracteres, numerales y guiones inferiores. Si se admiten caracteres adicionales en una implementación particular, entonces puede que ciertos caracteres deban ser codificados al visualizarlos para evitar confusiones. Esto es una responsabilidad de implementación de las herramientas.

Los nombres individuales de la jerarquía de un espacio de nombres separados mediante dos signos de dos puntos se pueden componer para formar un nombre de ruta.

nombre de clase

Cada clase debe tener un nombre no nulo único entre los clasificadores de su contenedor (paquete o clase contenedora). El alcance de un nombre es su paquete contenedor y los demás paquetes que pueden ver dicho paquete contenedor.

Véase nombre, donde se realiza una completa discusión sobre los nombres y sus reglas de unicidad.

Notación

El nombre de clase aparece en la parte o compartimento superior del rectángulo que representa a la clase. El compartimento del nombre también puede contener un nombre de palabra clave o de estereotipo y/o un icono de estereotipo y una lista de valores etiquetados entre llaves (Figura 13.143).

Opcionalmente se puede poner también una palabra clave de estereotipo entre comillas sobre el nombre de la clase, y/o un icono de estereotipo en la parte superior derecha del compartimento. El nombre de estereotipo no debe coincidir con una palabra clave predefinida.

El nombre de la clase aparece después, centrado horizontalmente y en negrita. Si la clase es abstracta, su nombre aparece en cursiva. Observe que cualquier especificación explícita acerca de su status de generalización (como *abstract* o *concrete*) tiene preferencia sobre el tipo de letra a utilizar en el nombre.

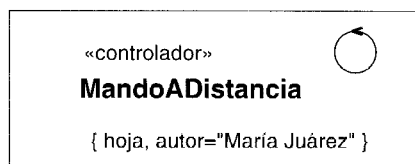
Debajo del nombre de la clase se puede situar de forma opcional una lista de cadenas que denotan sus propiedades (atributos del metamodelo o valores etiquetados). La lista puede mostrar también atributos de clase para los cuales UML no propone notación. Se pueden utilizar algunas palabras clave sin un valor concreto para denotar una determinada combinación de propiedad y valor. Por ejemplo, una clase hoja muestra la propiedad {leaf}, que es equivalente a {isLeaf=true}.

Por defecto, una clase mostrada dentro de un paquete se asume que está definida dentro de ese paquete. Para mostrar una referencia a una clase definida en otro paquete, se utiliza la sintaxis

Nombre-Paquete::Nombre-Clase

nombre de
estereotipo

valores
etiquetados



icono de estereotipo
(no es necesario especificar
ambos, icono y nombre
de estereotipo)

Figura 13.143 Compartimento del nombre